Transportation Research Part C 193 (2026) 105923



Contents lists available at ScienceDirect

Transportation Research Part C

journal homepage: www.elsevier.com/locate/trc



ELSSA: Explainable large language model-based decision support for public transit incident management

Jiahao Wang*, Amer Shalaby

Transit Analytics Lab (TAL), Department of Civil and Mineral Engineering (Transportation Engineering), University of Toronto, 35 St. George Street, Toronto, M5S 1A4, Ontario, Canada

ARTICLE INFO

Keywords:
Public transit operations
Incident management
Large language models
Retrieval-augmented generation (RAG)
Knowledge graphs
Decision support systems

ABSTRACT

Rapid incident response in public transit requires supervisors to make defensible decisions that align with both formal standard operating procedures (SOPs) and practical agency experience recorded in past reports. However, conventional retrieval-augmented generation (RAG) often fragments handbook context and underutilizes precedent knowledge embedded in unstructured control narratives. We present ELSSA, an Experience- and Logic-integrated System for Supervisor Assistance, which unifies rule-based and experience-based evidence for explainable incident-management support. ELSSA couples (i) a hierarchy-aware RAG pipeline over a structured ontology of the Toronto Transit Commission (TTC) incident handbook, and (ii) a knowledge-graph–based RAG module that converts TTC service occurrence reports into a regularized transit-incident knowledge graph via lightweight frequency-based auto-regularization; hybrid semantic and graph retrieval then surfaces relevant clauses and structurally similar precedents. Experiments using TTC supervisor training materials and Route 29 service occurrence reports show that ELSSA improves rule-based SOP question answering over strong RAG baselines and produces more robust experience-based control suggestions across original, radio-style, and paraphrased incident descriptions. A temporally separated 2023 held-out database further provides an initial test of generalization to newly received reports. A compliance analysis shows that service recovery and core documentation elements are highly visible in historical records which reflacts the reliability of the suggestions from ELSSA. Overall, ELSSA demonstrates how hierarchy-aware SOP retrieval and regularized incident knowledge graphs can support evidence-linked, auditable, and human-supervised decision assistance for public transit operations.

1. Introduction

Morning peak on a busy urban corridor, a bus–auto collision, an ill passenger, or a stalled vehicle in mixed traffic can all trigger the same operational reality for a public transit agency: service is disrupted, customers are waiting, and supervisors must make and justify decisions within minutes. Modern transit networks are large, tightly coupled socio-technical systems in which incidents propagate quickly across routes and modes. Each disruption demands rapid, defensible choices that balance safety, service reliability, and customer experience, while remaining consistent with agency policy and regulatory requirements.

In principle, on-road supervisors are supported by formal incident-management guidelines and standard operating procedures (SOPs) that codify “how the book says to do it.” In practice, effective incident handling also relies heavily on tacit, experience-based

* Corresponding author.
E-mail address: jhope.wang@mail.utoronto.ca (J. Wang).

https://doi.org/10.1016/j.trc.2026.105923
Received 24 December 2025; Received in revised form 10 June 2026; Accepted 7 August 2026
Available online 18 August 2026
0968-090X/© 2026 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

knowledge: how similar events actually unfolded in the past, which actions worked under local constraints, and which trade-offs were acceptable in context. Transit agencies increasingly struggle to sustain this blend of formal and experiential expertise. Two structural trends are particularly pressing: (i) institutional memory loss as senior staff retire or rotate, and (ii) rising incident volumes and variability, which push beyond the coverage of step-wise SOPs and training examples. These pressures create a growing gap between "what the handbook prescribes" and "what practitioners actually do" under dynamic field conditions.

At the same time, most archival operational knowledge remains locked in unstructured text: daily service occurrence reports, ad-hoc radio notes, investigation summaries, and post-mortem reviews. Conventional retrieval or template-based tools typically operate at the keyword level and struggle to surface the right precedent at the right time, often returning generic, weakly actionable advice. Large language models (LLMs) offer a powerful alternative, but naive prompting yields decontextualized suggestions that may neither reflect prior agency practice nor align tightly with official SOPs, raising concerns about trust, accountability, and safety in high-stakes deployments.

This paper addresses these gaps with **ELSSA**—an *E*xperience- and *L*ogic-integrated *S*ystem for *S*upervisor *A*ssistance—designed to turn historical incident records into explainable, experience-grounded decision support, complemented by situation-tailored rule-based SOP guidance. ELSSA integrates two complementary knowledge sources: (i) *rule-based* guidance from official incident-management handbooks and training materials, and (ii) *experience-based* precedents from curated service occurrence reports and action logs. Unstructured documents are converted into a transit-incident knowledge graph that encodes incidents, contexts, actions, and policy fragments as interconnected entities. On top of this representation, ELSSA employs agentic retrieval-augmented generation (RAG) to identify relevant precedents, cross-link them to applicable procedures, and generate recommendations with cited evidence and a traceable reasoning path. The resulting suggestions are concise, operationally specific, and auditable.

We instantiate and evaluate ELSSA using data from the Toronto Transit Commission (TTC), including supervisor training materials, and route-level service occurrence reports. The experiments reflect core elements of supervisor workflows: retrieving formal SOP guidance, identifying relevant incident-management procedures, and generating experience-based control suggestions from historical reports. For rule-based tasks, the hierarchy-aware RAG module improves SOP question answering relative to flat retrieval baselines, particularly for rewritten questions that require preserving handbook structure and procedural context. For experience-based recommendation, ELSSA is evaluated across three communication styles—the original report wording, a concise radio-communication tone, and a paraphrased narrative tone—as well as on a temporally separated 2023 held-out database. The results show that graph-structured retrieval improves action-code recovery, while auto-regularization reduces unsupported action fabrication and improves robustness to paraphrased and newly received incident descriptions.

Contributions. This paper makes three main contributions:

1. **Agentic, hierarchy-aware RAG pipeline for rule-based decision evidence.** We develop an agentic, hierarchy-aware RAG pipeline over a structured SOP information ontology, enabling an LLM agent to navigate sections and subsections of the incident-management handbook. This design improves the precision and efficiency of retrieving rule-based decision evidence compared with flat, chunk-based RAG.
2. **Agentic GraphRAG pipeline for explainable, experience-based decisions.** We propose an LLM-based knowledge-graph extraction and curation pipeline that builds a high-quality transit-incident knowledge graph, together with an automatic entity–relationship regularization module that enforces schema consistency. At query time, ELSSA combines hybrid search, graph queries, and LLM reasoning to (i) identify incident type and context, (ii) retrieve semantically and structurally proximate precedents and applicable procedures, and (iii) generate evidence-linked recommendations with a traceable reasoning path.
3. **Empirical study with TTC data.** Using TTC occurrence reports and supervisor training materials, we conduct a comprehensive evaluation of ELSSA under realistic incident-management scenarios, demonstrating improved accuracy on rule-based tasks and higher-quality, practitioner-aligned control suggestions compared with state-of-the-art RAG benchmarks.

Paper roadmap. Section 2 reviews related work on incident-management decision support, knowledge graphs, and RAG for high-stakes operations. Section 3 details the ELSSA pipeline, including system architecture and component methods. Section 4 presents the experimental design and results. Section 6 concludes with a discussion of implications, limitations, and directions for future research.

2. Related works

2.1. Disruption control and decision support in transit

Public transit agencies have long relied on real-time control actions to manage disruptions and irregular service. Conceptually, the literature separates into two layers: (i) *operational control actions* (the *what*) and (ii) *decision methods* used to select and parameterize those actions (the *how*). We review each layer in turn and conclude with implications for integrated decision-support systems.

Operational actions (the "what"). Agencies deploy a stable toolkit to stabilize service: *holding* to regulate headways, *short-turning/rerouting* to fill gaps, and *stop-skipping/deadheading* to recover time (Ceder, 2016). In higher-severity events they escalate to *rescheduling* and *resource reallocation*—retiming trips (Consilvio et al., 2021), reassigning vehicles or crews (Boyer et al., 2018), and *bus bridging* (Itani et al., 2020).

Decision methods (the "how"). Research focuses on selecting and parameterizing these actions under uncertainty. Beyond transparent rule-based dispatch, *mathematical programming* is widely used for timetable, vehicle, and crew decisions as well as corridor strategies (Mahmoudi et al., 2025); for example, Heidarigharehsoo and Saidi (2023) minimize generalized system cost across skip-stop and express–local designs. *Dynamic programming* and *model predictive control* (MPC) with rolling forecasts also appear in the literature;

e.g., Li et al. (2025) develop an MPC framework for rail signal control using ARIMA-based passenger-flow forecasts, and Feng et al. (2021) use DP for timetable rescheduling to reduce delays on a high-speed railway corridor. *Simulation-based metaheuristics* (e.g., genetic algorithms) coupled with fast LP/MILP feasibility checks are applied to non-convex, black-box search spaces; Wang et al. (2020) propose a multi-objective GA for dynamic bus rescheduling under traffic congestion. Meanwhile, *reinforcement learning* is increasingly used to derive state-dependent policies and is often combined with MPC/mathematical-programming layers for constraint handling; for instance, Ai et al. (2022) apply deep RL to timetable design to balance agency and passenger benefits, and Ai et al. (2026) incorporate LIME to improve the explainability of a deep Q-network.

In sum, real-time disruption response in bus and rail has progressed from manual heuristics to optimization-based rescheduling and bus bridging, yet deployments remain limited and operator judgment still dominates. As noted by Rodriguez et al. (2025), theoretical and experimental studies often rely on assumptions and rules that diverge from real-world conditions and constraints. Moreover, the *black-box* nature of many modern optimization methods (i.e., limited explainability) is a common barrier to adoption by transit agencies (Zdravković et al., 2025; International Transport Forum (ITF), 2025). Studies also consistently report gaps in (i) capturing and reusing *experience* across incidents, (ii) linking handbook guidance to on-the-ground actions, and (iii) providing *auditable* justifications during incidents (Atkinson et al., 2024; Yan and Hunter, 2023).

2.2. Large language model–based approaches

Large Language Models (LLMs) are multi-layer Transformer systems (Vaswani et al., 2017) pre-trained on large corpora (Dwivedi et al., 2023) to acquire broad linguistic competence and world knowledge. Beyond generic NLP tasks (e.g., sentiment and semantic analysis Zhang et al., 2023b; Xu et al., 2024b), recent work shows LLMs are effective for knowledge-intensive applications such as question answering, relation and event extraction, and contextual reasoning (Xu et al., 2023).

Within transportation, LLMs have been explored for domain Q&A (Syed et al., 2024), infrastructure planning (Roberts et al., 2023), project operations (Abbas et al., 2023), maintenance and safety (Rane, 2023; Du, 2024), and multimodal perception (Ashqar et al., 2024). Parallel efforts emphasize building data ecosystems and benchmarks to evaluate domain readiness (Papageorgiou et al., 2024), including TransportBench for reasoning (Syed et al., 2024), curated QA sets (Zhang et al., 2023a), image datasets for safety inspection (Prajapati et al., 2024), and GTFS-centric tasks that probe semantic retrieval and code generation (Devunuri et al., 2024). These trends signal a shift from PoC chatbots toward domain-focused assistants that can interface with heterogeneous transit data streams. However, we noticed that existing LLM applications in transit primarily target *customer information, analytics, or planning*. These systems improve accessibility and analysis but rarely address *supervisor decision support* during live incidents, where actions must align with both *rule handbooks* and *experience precedents*, and where decision paths must be auditable. On the other hand, according to Du (2024), most LLM applications *do not* enforce domain-grounded provenance (explicitly citing the source of facts) or operational constraints (feasibility, rule compliance). This limits trust and auditability for decision support in control rooms.

Two complementary families of improving LLM adaptations are prevalent. *Fine-tuning* specializes a base LLM to a target domain or task. For modest models such as BERT or RoBERTa, end-to-end supervised fine-tuning remains feasible; for example, Wang and Shalaby (2024) fine-tune BERT for several transit-focused social-media NLP tasks. As the size of modern LLM families has grown, *parameter-efficient fine-tuning* (PEFT) has become the norm: the backbone is frozen and only a small set of adapter/low-rank parameters is learned, substantially reducing compute and memory. For instance, Wang et al. (2024) fine-tune the open-source multimodal *VisualGLM* on a textual transportation corpus (STD; 12.5M tokens) to strengthen transportation-domain knowledge; similarly, Zheng et al. (2023) adapt *LLaMA-3* with the *TrafficSafety-2K* dataset—curated from government guidance and LLM-generated instruction–output pairs—to improve performance on transportation-safety queries. Despite these gains, fine-tuning pre-trained LLMs still requires curated, high-quality data and nontrivial computational resources, which can limit frequent updates and rapid domain drift handling.

Retrieval-Augmented Generation (RAG) system, on the other hand, offers a complementary, lightweight path by retrieving domain-specific evidence at inference time (Lewis et al., 2020). Recent variants extend basic dense retrieval: *DH-RAG* incorporates dynamic long/short-term memory to preserve historical context (Zhang et al., 2025), while *SELF-multi-RAG* allows models to decide *when* to retrieve and *how* to reformulate queries based on conversational cues (Roy et al., 2024). Beyond text-only retrieval, knowledge-graph–aware RAG (*GraphRAG*) converts unstructured corpora into graph structures that encode entities and relations, enabling multi-hop reasoning and better scaling to large collections (Edge et al., 2024; Xu et al., 2024a). Concrete systems include *MedGraphRAG*, which integrates KG retrieval into the generation loop to guide domain reasoning and improve factuality (Wu et al., 2024); *KRAGEN*, which retrieves subgraphs and uses graph-of-thoughts prompting to support relational reasoning—reducing hallucinations by roughly 20–30% (Matsumoto et al., 2024); and *Dual-Pathway KG-RAG*, which runs structured graph retrieval alongside unstructured text retrieval, delivering more consistent biomedical QA and cutting hallucinations by about 18% (Zhu et al., 2025). Collectively, these results indicate that tailoring RAG to domain knowledge—especially via KG representations—can significantly enhance accuracy, faithfulness, and scalability in specialized applications.

Through literature review, we noticed that, in transit specifically, we hardlly see any state-of-the-arts on utilizing LLM to provide transit agency a talied, reasonning, comprehensive incident control suggestions based on multi-sources inputs. No comprehensive LLM-based pipeline is provided for such usecase. Meanwhile, the practice on LLM and domain-specific information combination are limitated: systems may retrieve text but lack deeper multidimentional correlation structure, lack of systematic evaluation on the current possible solutions.

To bridge the gaps, we proposed ELSSA by (i) *integrating* rule handbooks and experience precedents into a unified *transit-incident knowledge graph*, (ii) implementing *agentic GraphRAG*—hybrid KG / semantic retrieval plus tool-use—to jointly retrieve policy fragments and similar cases, and (iii) generating *evidence-linked, policy-consistent* recommendations whose reasoning path is traceable and

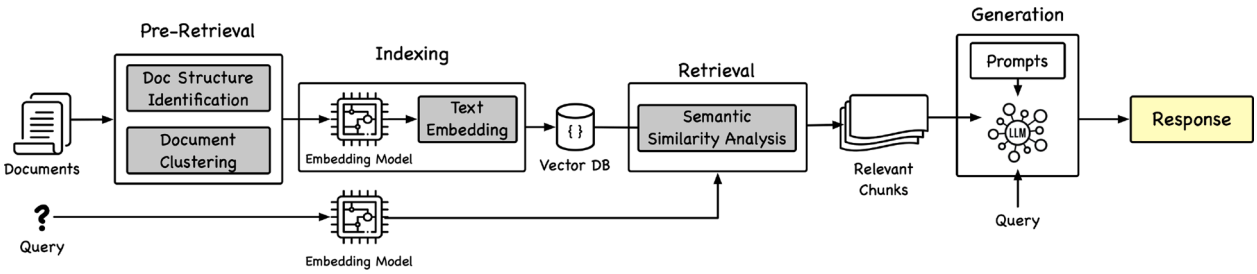


Fig. 1. Baseline naive RAG pipeline used as a comparison point for ELSSA. Documents are pre-processed (Structure identification and clustering), embedded and stored in a vector database. At inference time, the query is embedded, semantically similar chunks are retrieved, and these chunks are appended to the LLM prompt to generate a response.

whose feasibility is screened by operational constraints. Concretely, ELSSA retrieves and cites the *specific* handbook clause and the *most similar* historical incident report, explains the linkage, and outputs actions.

3. Methods

This section presents the ELSSA pipeline, with details about the hierarchy-aware agentic RAG system, the knowledge-graph-based agentic RAG, and how these components are combined to leverage both official training materials and past incident control reports. Together, they provide experience-based and rule-based control suggestions that reflect the needs of a real-world transit agency.

3.1. Injecting domain-specific knowledge with retrieval-augmented generation (RAG)

We use a retrieval-augmented generation (RAG) framework to bring TTC-specific knowledge into the LLM's decision process. Two sources are indexed: (i) the rule-based incident standard operating procedures (SOPs) used by on-road supervisors, and (ii) historical incident control reports recorded by Transit Control. At inference time, the system retrieves relevant SOP sections and past cases so that the LLM's suggestions follow TTC practice rather than generic rules.

The baseline "naive" RAG pipeline (Fig. 1) follows the common three-stage design: documents are split into small text chunks and embedded in a vector space; the supervisor's query is embedded with the same model; the most similar chunks are retrieved and appended to the LLM prompt as external context. Although simple and effective, this design exhibits several weaknesses in our incident-management setting:

- **Limited understanding of complex incidents.** Supervisors often describe situations in short, informal phrases that implicitly bundle several steps (safety checks, detours, communications, etc.). A chunk-level retriever that only matches local semantics often returns isolated paragraphs that do not collectively cover the full procedure chain required for the incident.
- **Loss of SOP structure and functional context.** The incident management handbook has a clear hierarchy (sections, prerequisites, cautions, follow-up actions). When it is split into small chunks, safety notes or critical preconditions may be retrieved on their own, without the surrounding context that explains when they apply, creating a risk of incomplete or misleading guidance.
- **Underuse of structured incident data.** Historical incident reports include labelled fields (e.g., incident type, event type, route, time). Naive RAG largely ignores these relationships and relies only on text similarity. In our relatively small incident database, purely semantic matching is often insufficient to find good analogues; we need to exploit these labels to widen the candidate set and surface structurally similar cases, not just lexically similar ones.
- **Limited reach to related information.** Once a small set of high-scoring chunks is retrieved, naive RAG rarely explores semantically adjacent but non-overlapping material (e.g., prerequisite sections, follow-up procedures, or related incident types on the same corridor). As a result, supervisors may receive answers that address only part of the situation and omit relevant cautions, alternatives, or historical precedents that lie in nearby handbook sections or related incident reports.

These limitations indicate that a simple chunk-and-retrieve setup is not sufficient for transit incident management. We require a RAG design that is aware of the operational context, preserves more of the SOP hierarchy, and leverages structured incident metadata so that the LLM receives a compact, coherent set of TTC-specific evidence tailored to each supervisory decision.

To address these limitations and obtain higher-quality evidence for LLM-generated incident management suggestions, we design two complementary RAG modules:

- **Agentic, hierarchy-aware RAG over the SOP handbook.** This module focuses on retrieving rule-based guidance from the official TTC incident management handbook. An LLM-powered agent first interprets the supervisor's query (e.g., incident type, affected mode, escalation level) and then navigates a hierarchy-aware index of the SOP (sections, subsections, prerequisites, cautions). Instead of pulling isolated paragraphs, the agent retrieves small, coherent bundles of procedures that preserve critical context (safety notes, "must-do-before" steps, follow-up actions) while keeping the total context size manageable for operationally deployment.

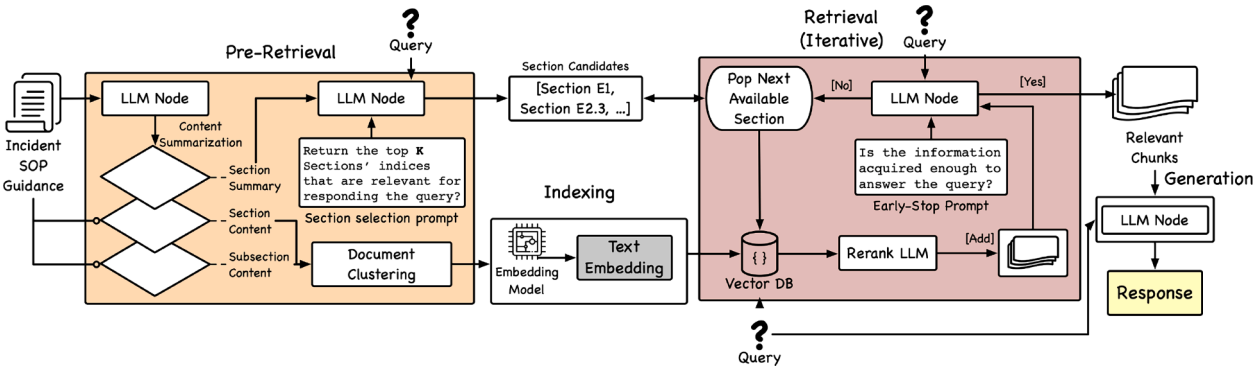


Fig. 2. Agentic, hierarchy-aware RAG over the TTC incident management handbook. An LLM agent summarizes and clusters SOP sections, selects a small set of candidate sections for a given query, and then guides an iterative retrieval process with early stopping, providing the generator LLM with compact, structurally coherent evidence.

- **Knowledge-graph-based RAG over historical incident reports.** This module treats past incident reports as an "experience database." We construct a knowledge graph where nodes represent incidents, routes, locations, times, causes, and actions taken, and edges capture relationships derived from both structured fields (e.g., incident type, event type) and free-text descriptions (what happened, how it was handled). At query time, we use hybrid search to traverse the knowledge graph and surface families of related cases—similar incident types on the same corridor, comparable operating conditions, and actions that led to successful resolution. This graph-based retrieval extends beyond pure text similarity, improves coverage in a relatively small incident corpus, and provides the LLM with a compact set of diverse, structurally similar examples.

3.1.1. Agentic-based hierarchy-aware RAG over the SOP handbook

To improve the quality of incident control suggestions grounded in the TTC incident management handbook, we adopt an agentic, hierarchy-aware RAG architecture that explicitly leverages the structure of the SOP document. The system, illustrated in Fig. 2, consists of two main stages: (i) a pre-retrieval stage for query-aware section selection, and (ii) an iterative, structure-aware retrieval stage.

In the pre-retrieval stage, we embed an LLM-based agent inside the RAG loop. During offline handbook preparation, instead of directly splitting the document into fixed-size chunks, an LLM node generates a short summary for each section and subsection. These summaries serve as high-level descriptors for subsequent decision making. The content of each section and subsection is then segmented into smaller chunks but stored together with its parent section identifier, enabling hierarchy-aware retrieval.

Given a supervisor's query, in the pre-retrieval stage, the agent is provided with both the raw query text and the set of section and subsection summaries. Its task is to infer the operational context, rank the sections, and select the top-k candidate sections that are likely to be relevant, while optionally justifying its choices. This step focuses the search on a small set of meaningful regions in the handbook rather than treating the entire document as a flat collection of chunks.

In the retrieval stage, the retriever operates over a hierarchy-preserving index of the SOP (section → subsection → content chunk) rather than over uniformly sized fragments. For each agent-selected section, the retriever incrementally pulls coherent blocks of text that preserve local ordering and context (e.g., prerequisites, safety notes, follow-up procedures), guided by semantic similarity to the query. To further improve efficiency, we adopt an early-stop mechanism: after each retrieval step, an LLM-based decision node inspects the newly retrieved content and determines whether the accumulated evidence is sufficient to answer the query or whether additional sections or subsections should be explored. This agent-in-the-loop mechanism introduces an explicit termination decision instead of always filling the context window to capacity.

Once the agent declares that enough information has been gathered, the generator LLM produces incident control suggestions based on the original query and the curated set of SOP excerpts. Compared with traditional RAG that splits documents into fixed-size chunks and reassembles them purely by embedding similarity, the hierarchy-aware design preserves the original structure and sequencing of the handbook and allows content selection to be guided by an understanding of procedural context. This reduces the risk of omitting critical prerequisites, reordering steps, or inducing unsafe workflows, while still operating within the tight latency and context constraints of operationally deployable.

3.1.2. Knowledge-graph-based RAG over historical incident reports

To exploit the incident experience embedded in past operations, we adopt a knowledge-graph-based (KG-based) RAG module (based on the GraphRAG structure, first proposed by Microsoft in Edge et al., 2024) over historical TTC incident management reports. A high-level abstraction of how the KG-based RAG is built and how the information query is performed are shown in Fig. 3, As illustrated, unstructured incident records are first converted into a domain-specific knowledge graph; this graph is then indexed and traversed to retrieve families of similar cases and proven responses as context for the LLM with hybrid query.

Entity and relationship extraction with an LLM-guided schema. The original incident database is stored in tabular form with four key fields: *Incident Type*, *Event Type*, *Nature of Problem*, and *Actions Taken*. At a high level, *Incident Type* describes the broad category

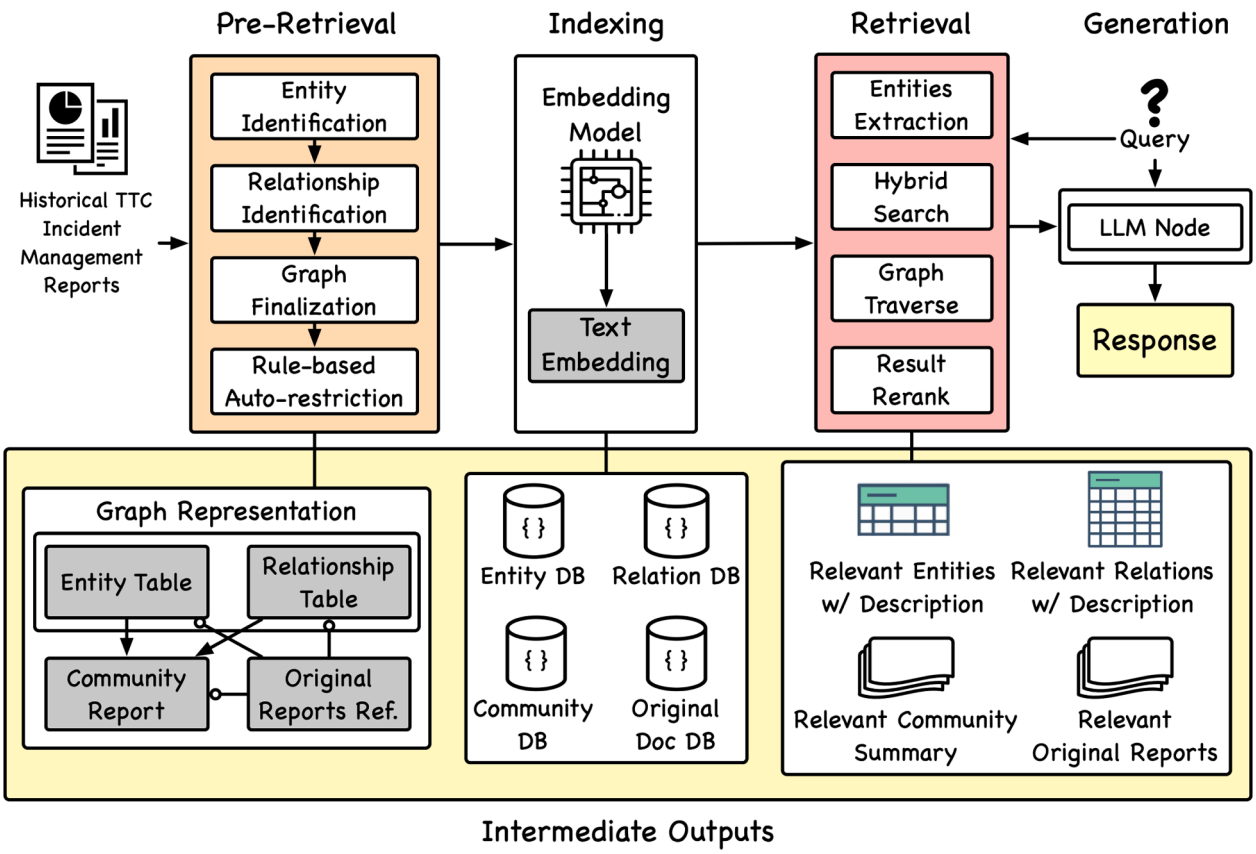


Fig. 3. Knowledge-graph-based RAG module over historical TTC incident management reports. LLM-guided graph extraction and rule-based auto-regularization produce normalized entity and relation tables, community-level summaries, and links to the original reports. These graph representations are embedded and stored in dedicated databases, enabling hybrid, graph-aware retrieval and reranking at query time before response generation.

of the incident (e.g., mechanical issue), *Event Type* specifies a more concrete manifestation (e.g., mirror damage, tire issue, door malfunction), *Nature of Problem* is a free-text description written by field staff or control center personnel, and *Actions Taken* documents the operational response (e.g., change-off, tow, private to garage). While these four columns already provide useful structure, the logical connections between them are shallow: they do not explicitly encode causes, impacts, resource requirements, or cross-incident patterns.

To obtain a richer, decision-oriented representation, we employ an LLM-based extractor governed by a strict, prompt-defined schema (Fig. 4). For each record, the extractor maps the four fields into a small set of canonical entities and relations tailored to transit operations. Specifically, we define the following entity types:

- **INCIDENT_PATTERN** and **EVENT_CATEGORY**: instantiated from the incident and event class labels in the original record.
- **ROOT_CAUSE**: the underlying factor or component that caused the incident, inferred from the *Nature of Problem*.
- **ACTION_STRATEGY**: the operational actions taken to address the incident (e.g., change-off, tow, short-turn), inferred from *Actions Taken*.
- **RESOURCE_REQUIREMENT**: specific resources required to implement the action strategy (e.g., tow truck, replacement bus, supervisor support).
- **SERVICE_IMPACT**: the impact on service, inferred from both *Nature of Problem* and *Actions Taken* (e.g., out-of-service duration, service gap).
- **LOCATION_PATTERN**: the locations where the incident occurred or was addressed (e.g., corridor, station, division, garage), extracted primarily from the *Nature of Problem* and sometimes from *Actions Taken*.
- **MEASUREMENT**: explicit time durations or other quantitative indicators mentioned in the record (e.g., `15_Min` out-of-service).

All entities are normalized to deterministic, uppercased names (e.g., `OUT_OF_SERVICE`, `CHANGE_OFF`, etc.) to ensure consistency across records.

Relationships between entities are constructed according to a fixed policy that reflects operational reasoning:

- **Scope linkage:** `EVENT_CATEGORY` nodes are connected to their corresponding `INCIDENT_PATTERN` nodes via *in_scope* edges.

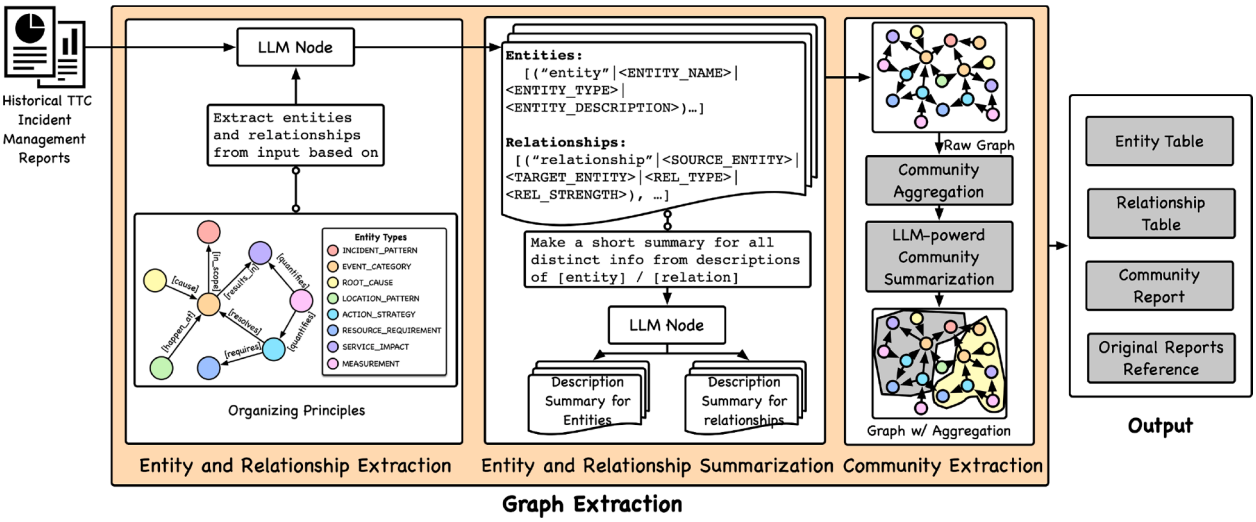


Fig. 4. LLM-guided graph extraction from historical TTC incident management reports. The extractor maps tabular and free-text fields into typed entities and relations according to an operational schema, summarizes their descriptions, and aggregates them into communities, yielding normalized entity and relationship tables, community reports, and references to the original reports.

- **Causal structure:** `ROOT_CAUSE` nodes are linked to `EVENT_CATEGORY` nodes with *causes* edges.
- **Impact propagation:** `EVENT_CATEGORY` nodes are linked to `SERVICE_IMPACT` nodes using *results_in* edges.
- **Resolution strategies:** `ACTION_STRATEGY` nodes are connected to `EVENT_CATEGORY` nodes via *resolves* edges.
- **Resource dependencies:** when specific resources are mentioned, `ACTION_STRATEGY` nodes are linked to `RESOURCE_REQUIREMENT` nodes with *requires* edges.
- **Quantification:** when explicit durations appear, `MEASUREMENT` entities are linked to the corresponding `SERVICE_IMPACT` or `ACTION_STRATEGY` nodes via *quantifies* edges.
- **Edge strength and de-duplication:** each edge is assigned a discrete strength score (1–10) indicating how explicitly the relationship is stated in the record, supporting calibrated similarity search. Entities are de-duplicated across records by their `(type, name)` pair, and the strongest associated relationships are retained.

The extraction prompts combine explicit schema instructions with representative examples to guide LLM inference and reduce spurious entities and relationships.

After entities and relationships are extracted, each node retains links to the original records from which it was derived. An additional LLM-based summarization step produces a short canonical description for every entity and relation. The summarizer is instructed to merge all distinct information from multiple mentions into a single concise description, so that each node and edge has a compact textual representation for downstream retrieval.

The resulting entities and relationships form a consolidated property graph that acts as an operational "experience base" for incident management. Nodes correspond to recurring patterns (incident types, event categories, causes, actions, impacts, locations, resources), and edges encode how these patterns co-occur and interact in real operations. Compared with the original table, this representation is substantially more expressive: it reveals, for example, that a given mechanical pattern recurrently leads to `OUT_OF_SERVICE` or `SERVICE_GAP` on specific corridors, and that particular action strategies and resource deployments are typically used to restore service.

Finally, entities and relationships are aggregated into communities using the Leiden algorithm (Traag et al., 2019), forming a multi-layered hierarchical information structure in which communities are groups of nodes that are densely connected to each other but sparsely connected to the rest of the graph. For each community, an LLM-based summarization step produces a "community report" that highlights common patterns and distinctive features across its members. These reports provide a macro-level view of the graph and enable the retrieval module to quickly navigate towards promising regions of the experience base.

Rule-based auto-regularization of entities and relations. We propose a lightweight post-processing procedure that jointly regularizes relation instances and infers missing entity types using only empirical frequency information (pseudo-code is provided in (Algorithm 1) in Appendix A).

To reduce manual annotation effort and improve the internal consistency of the LLM-extracted knowledge graph, we deploy a lightweight frequency-based post-processing procedure that jointly *regularizes* relation instances and *infers* missing entity types from observed relations:

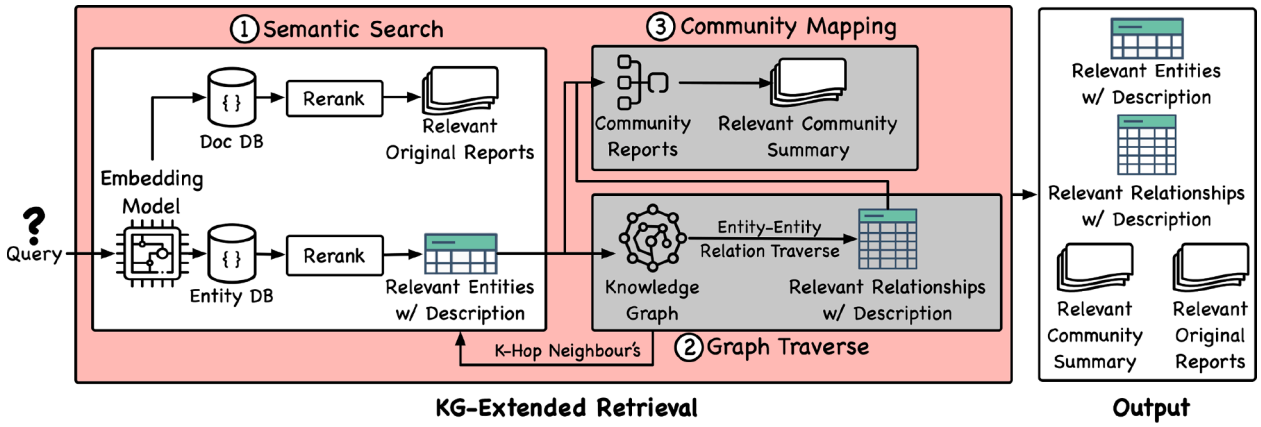


Fig. 5. KG-extended hybrid retrieval at query time. A supervisor query is embedded and used for (1) semantic search over the document and entity databases, (2) graph traversal from the most relevant entities to their k-hop neighbors and connecting relations, and (3) community mapping to retrieve community-level summaries. The resulting entities, relations, community reports, and original incident reports provide structured, experience-based evidence for the downstream LLM.

1. **Canonical type signatures per relation description.** Each binary relation instance has a free-text description and two endpoints. When an endpoint has a known type, it is attached as `source_type` or `target_type`. For each relation description, we compute the empirical distributions of source and target types and take the most frequent pair as its *canonical type signature*.
2. **Canonical direction per description–type pair.** Directional inconsistencies (e.g., "A caused B" vs. "B caused by A") are then resolved using instances where both endpoint types are known. For each relation description and unordered type pair $\{T_1, T_2\}$, we count how often $T_1 \to T_2$ and $T_2 \to T_1$ occur, and adopt the majority orientation as the *canonical direction* for that description–type pair.
3. **Direction enforcement and endpoint-type imputation.** We traverse all relation instances and, whenever both endpoint types are known, swap source and target if the observed orientation disagrees with the canonical direction. We then use canonical type signatures to impute missing endpoint types: missing target types are always filled when a canonical target type exists, while source types are imputed only when the canonical source type is sufficiently frequent. This yields a relation table with consistent directions and improved endpoint-type coverage.
4. **Entity-type inference via majority voting.** Using the regularized relations, we infer types for entities that remain unlabeled. For each entity, we aggregate votes from all incident relations in which it appears: every occurrence as a source (target) with `source_type` (`target_type`) $= T$ adds one vote for type T. An unlabeled entity is assigned the type with the highest vote count, provided this maximum exceeds a threshold `min_votes`; otherwise, it remains unlabeled. This acts as a simple label-propagation mechanism from confidently typed to initially unlabeled entities.

The procedure is computationally inexpensive, dominated by grouping and counting over relation instances, and scales approximately linearly with the number of relations while substantially improving type coverage and internal consistency.

Knowledge-graph-based hybrid retrieval for RAG. As illustrated in Fig. 5, at query time the supervisor's free-text description is embedded and used to drive a three-stage KG-extended retrieval process.

First, in the *semantic search* stage, the query embedding is used to perform two parallel searches: a document-level search over the original incident reports and an entity-level search over the entity database. A reranking model selects a small set of top candidate reports and entities whose descriptions are most semantically similar to the query.

Second, in the *graph traverse* stage, the retrieved entities are treated as seeds for exploring the knowledge graph. We perform k-hop neighbor expansion and traverse entity–entity relations to (i) extend the coverage of relevant entities and (ii) collect the relationships along the traversed paths. This yields a denser set of graph evidence that captures how causes, impacts, actions, and resources co-occur in similar operational contexts.

Third, in the *community mapping* stage, seed and neighbor entities are mapped to their corresponding communities, and the associated community reports are retrieved. These community-level summaries provide a higher-level view of recurring incident patterns and typical resolution strategies, complementing the fine-grained entity and relation descriptions.

By shifting from flat, record-level retrieval to knowledge-graph-based hybrid retrieval, this module extends the reach of RAG beyond pure text similarity. It improves recall in a relatively small incident corpus, preserves the relational structure between causes, impacts, and responses, and supplies the LLM with concise, interpretable evidence drawn from past TTC operations rather than a long list of loosely related text fragments.

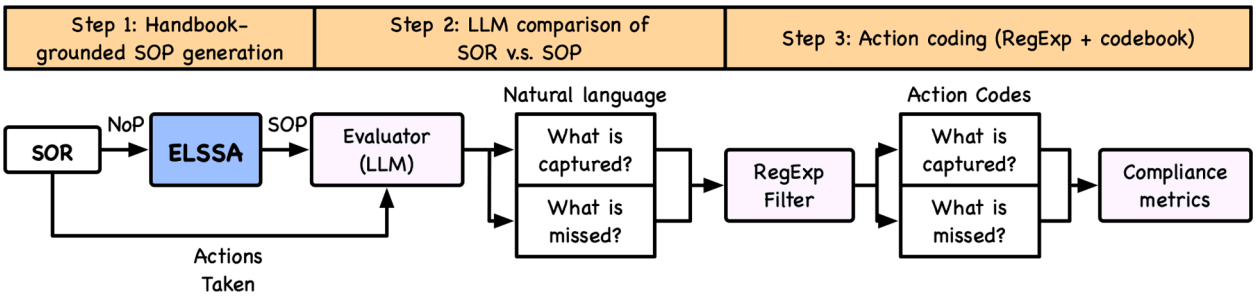


Fig. 6. Overview of the ELSSA-based SOP compliance evaluation pipeline. For each of the 54 TTC service occurrence reports (SORs), ELSSA ingests the Nature of Problem field and produces a handbook-grounded, step-by-step SOP (Step 1). An independent evaluator LLM then compares this SOP against the Actions Taken narrative, classifying each recommended step as either captured or missing/undocumented in the SOR (Step 2). The resulting natural-language assessments are converted, via regular-expression filtering and a handbook-derived codebook, into structured action codes, from which code-, category-, and overall-level compliance metrics are computed (Step 3).

3.2. Comparison of ELSSA rule-based SOP suggestions with real-world incident reports

Building on ELSSA's ability to generate tailored SOP guidance from free-text incident descriptions, we next evaluate how these rule-based suggestions compare with the actions documented in real-world TTC incident reports. Crucially, our redesigned evaluation separates three phenomena that a naive "followed vs. missing" comparison conflates: (i) a *source-supported SOP recommendation* (an action ELSSA recommends *and* that is grounded in the retrieved TTC handbook evidence for that incident); (ii) a *SOR-documented execution* (the real-world report explicitly records an action that operationally matches the SOP item); and (iii) an *LLM over-prescription* (an action the model suggests but that the retrieved SOP evidence does not support, or that is not clearly applicable to the incident).

We narrow the analysis to incidents whose *Incident Type* is *Investigation* or *Cleaning*, drawn from both the previously cleaned Route 29 collection and the temporally separated 2023 test set. Events under these two types correspond closely to the generic incident-management scenarios covered in the TTC Emergency Response Handbook (e.g., collisions, disorderly customers, emergency-service interactions). They therefore provide a natural testbed for comparing handbook-prescribed procedures with actual supervisor behaviour.

A key caveat is that the SORs we analyse are a specific subset of the documentation associated with an incident: they summarize the supervisor's actions to stabilize the situation and restore service, but additional reports (e.g., divisional follow-up, safety investigations, employee forms) may exist. For the purposes of this study, we *assume* each SOR is the only available account of field actions. This assumption allows us to treat missing items as potential gaps in either procedure or documentation and to demonstrate how ELSSA could highlight such gaps in practice. We interpret our results as indicative patterns rather than definitive judgments on individual incidents.

To ensure reliability of the derived compliance metrics, we apply two validation steps:

1. **Pattern-matching verification**: We manually review a 20% random sample of incidents to confirm that action phrases in the SORs are correctly mapped to action codes.
2. **Category consistency checks**: We cross-check all code–category assignments against the TTC SOP handbook to ensure that each action is grouped under the appropriate operational dimension.

For each of the incidents, we then run the following pipeline, as illustrated in Fig. 6:

1. **SOP suggestion generation:** We provide ELSSA only with the *Nature of Problem* field and request a rule-based, step-by-step response grounded in the TTC incident-management handbook (i.e., without using historical SORs).
2. **LLM-based comparison:** A separate evaluation LLM receives (i) the original *Actions Taken* text from the SOR and (ii) ELSSA's generated SOP. It is instructed to identify, for each recommended SOP step, whether it is (a) clearly followed and documented in the SOR, or (b) missing or not documented.
3. **Action coding:** The natural-language comparison output is parsed into a structured list of actions and mapped onto a pre-defined coding scheme derived from the TTC SOP handbook (Appendix E). The scheme comprises discrete action codes organized into seven supervisor-relevant categories (e.g., communication and coordination, scene management, investigation, safety/medical response, service operations, evidence and documentation, and support/aftercare).

After coding, we compute compliance metrics at the code, category, and overall levels. For a given unit of analysis (e.g., a category within an incident, or across all incidents), we define the compliance rate as

$$\text{Compliance Rate} = \frac{\text{Items Followed}}{\text{Items Followed} + \text{Items Missing}} \times 100\%.$$

We report this metric at two levels:

- **Category level:** Compliance aggregated over all codes belonging to each of the seven supervisor-relevant categories, highlighting which operational dimensions (e.g., communication vs. investigation) tend to be well-documented or systematically under-documented.
- **Overall level:** Compliance aggregated over all supervisor-relevant categories, providing a global measure of how closely SORs, viewed in isolation, align with the handbook-derived SOPs generated by ELSSA.

Because the evaluation pipeline is stochastic, we repeat the complete generation–extraction–comparison procedure across multiple runs. Stochasticity arises from both the LLM-based components of ELSSA, including retrieval stopping and response generation, and the downstream LLM-based extraction and judging steps. To quantify this variability, we vary the generation temperature (0.0, 0.2, and 0.4) and random seed, and report the mean and standard deviation for each metric.

On the other hand, ELSSA uses `gpt-4o-mini` as the generation model, while the primary action-presence judge uses a stronger and different model, `gpt-5-mini`. Using a separate judge model helps reduce self-enhancement bias that could arise if the same model family were used to both generate and evaluate the recommendations. As an additional robustness check, we repeat the comparison with a second judge, `gpt-4o-mini`, and report cross-model agreement using Cohen's κ. We also report the repeated-run self-consistency of the primary judge using Fleiss' κ, computed over recommendations that appear in all repeated runs.

Interpreted together, these measures allow us to assess not only whether ELSSA's rule-based suggestions are consistent with TTC procedures, but also where real-world incident reports systematically omit or under-emphasize SOP-consistent actions that ELSSA surfaces as relevant.

4. Experiments

In this section, we systematically evaluate ELSSA in realistic operational scenarios using real-world TTC data. We first assess the two RAG modules separately within their respective use cases: (i) the agentic, hierarchy-aware RAG for handbook information retrieval, by measuring how accurately it answers questions derived from the TTC Emergency Response Handbook; and (ii) the knowledge-graph-based RAG for experience-based suggestions, by comparing its recommended actions with those documented in historical incident reports. Finally, we analyze how ELSSA's rule-based SOP suggestions align with real-world supervisor reports, quantifying SOP compliance and identifying systematic patterns of over- and under-documentation.

4.1. Dataset

The experimental dataset comes from the Toronto Transit Commission (TTC) (system map shown in Fig. 7), the public transit agency responsible for bus, streetcar, and subway services across Toronto. According to the TTC's 2025 Asset Management Plan, the agency currently operates more than 160 bus routes, eleven streetcar routes, and three rapid-transit lines.[1] The network is therefore large and multimodal, with bus routes connecting outlying neighbourhoods to downtown streetcar corridors and subway trunk lines. This operating environment produces a diverse range of incident types—mechanical failures, road blockages, passenger emergencies, and more—that on-site supervisors must handle effectively.

The first component of our dataset is the official TTC Emergency Response Handbook and its associated training quizzes. The handbook contains high-level guidelines, standard operating procedures (SOPs), and job aids that form the basis of rule-based decision support. It sets out general incident-response steps (receive the call, attend the scene, evaluate, update, investigate, clear, and follow up), while also covering specific hazards such as medical emergencies, fires, derailments, and environmental spills. These documents provide the normative guidance that we embed in the hierarchy-aware RAG module described earlier.

The quiz set consists of 56 assessment questions extracted from the training materials. Each question is tied to a particular handbook section. A summary of the question distribution by section, along with several example items, is provided in Table 1.

Because some original quiz items are highly abbreviated and lack situational context, we prepare an additional rewritten version of selected questions (Fig. 8). The rewritten items add brief background scenarios and adjust the tone to better resemble how supervisors would describe incidents in practice, while preserving the same underlying handbook content. This allows us to evaluate the hierarchy-aware RAG's ability to retrieve and apply SOP content under both structured quiz formulations and more natural, scenario-style queries that better reflect real TTC operations.

The second component of the dataset is a set of service occurrence reports (i.e., incident management reports) collected by the TTC. We focus on reports for bus Route 29, which TTC staff identify as one of the most operationally challenging routes in the network. The dataset covers incidents recorded in **November 2024 (2024 DB)** and, for our experiments, we use four informative fields to construct the experience-based knowledge graph: *Incident Type*, *Event Type*, *Nature of Problem*, and *Actions Taken*. An example record is shown in Fig. 9. Overall, the dataset contains 187 records spanning 52 distinct event types across 9 incident types. The distribution of records by incident type is shown in Fig. 10, and detailed incident–event type counts are reported in Appendix C.

To evaluate the proposed decision-support system, we use 5-fold cross-validation rather than a single train/test split, so that every record serves as a test query exactly once while the remaining records form the knowledge base for that fold. We stratify the folds by *Incident Type* to preserve the overall incident-type distribution in each test set. Because some incident types are sparsely observed, we keep the records of the rarest class (*Mechanical - VISION*, 3 records) in the knowledge base across all folds, ensuring that each incident type retains at least one representative record that remains retrievable. This procedure yields approximately 150 records for

[1] https://www.toronto.ca/legdocs/mmis/2025/ttc/bgrd/backgroundfile-254421.pdf

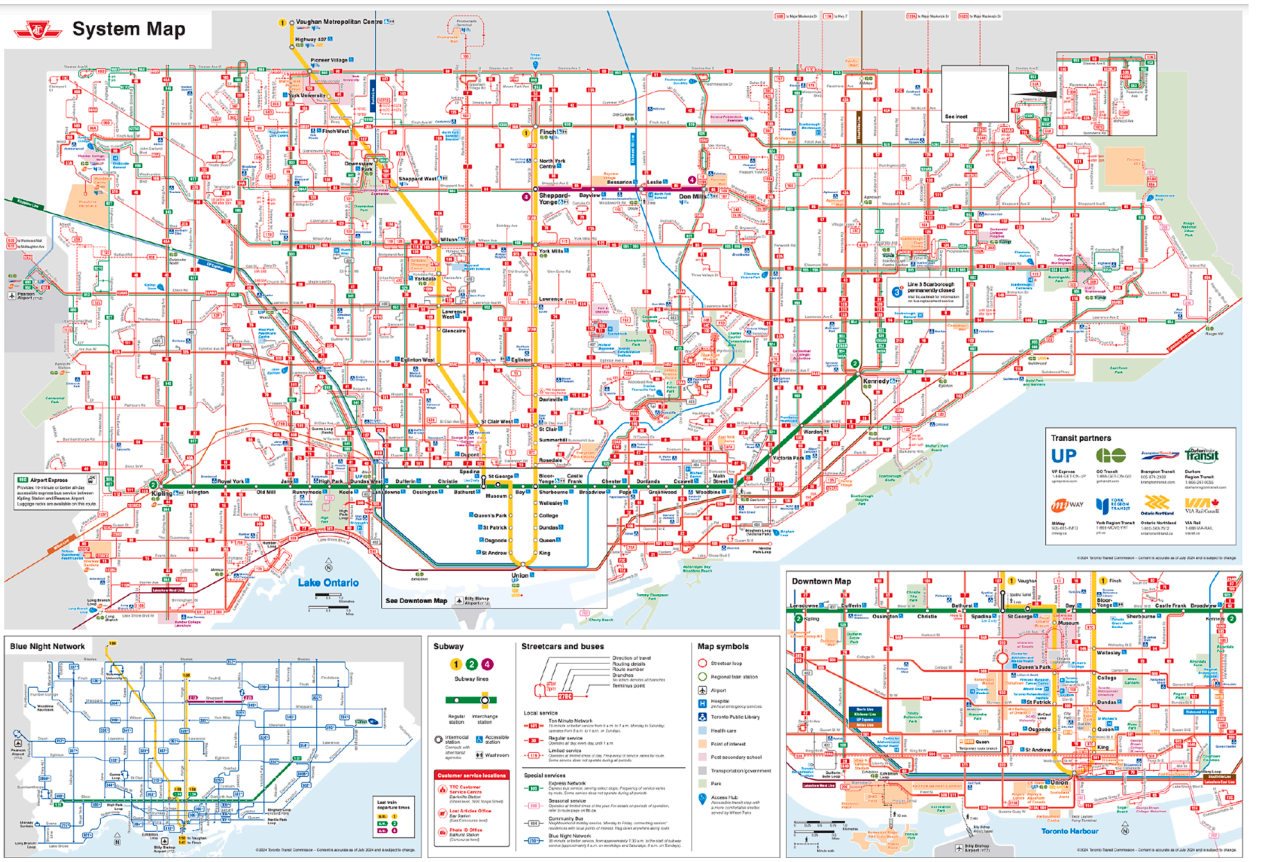


Fig. 7. Toronto transit commission (TTC) system map. The network combines bus, streetcar, and subway services across the City of Toronto, creating a diverse set of operating environments and incident types.

Table 1
Distribution and examples of assessment questions by handbook section. Each row lists the question type, count, and a representative item.

Section Code	Section Title	Question Type	No. Qs	Example / Quick View
E1	Guidelines for Responding to Incidents	Sequence matching	1	Match incident response steps to the correct guideline (A–G).
E2_1	Radio Communications Protocol	Fill in the blank	5	Complete: "Acknowledge receipt of a radio transmission by saying _____."
E2_2	Writing Reports	True/False	7	Should contributing factors be included in an incident report?
E3_1	Responding to Medical Emergencies and Assaults	True/False	5	Always wear latex gloves when arriving first at a medical emergency.
E3_2	Responding to Collector Booth Alarms	True/False	4	Do not wait for police before responding to a station collector alarm.
E3_3	Responding to Fire, Smoke, and Burning Odour	Fill in the blank	6	Identify the exact _____ and _____ of fire, smoke, or odour.
E3_4	Responding to a Fire Alarm	Fill in the blank	4	Upon arrival, _____ the arrival of emergency services if smoke or fire is evident.
E3_5	Responding to a Bomb Threat	True/False	5	Remove a suspicious package from TTC property immediately.
E3_6	Responding to a Streetcar Derailment	Fill in the blank	5	Determine if there is _____ of immediate _____ to employees, customers, and police.
E3_8	Responding to a Community Emergency	Fill in the blank	3	Assess what _____ the situation may have on TTC _____ or property.
E4_1	Conducting an Emergency Evacuation	True/False	4	Evacuation time is not a factor when deciding whether to evacuate.
E4_2	Controlling an Environmental Spill	Sequence matching	1	Order the steps: secure vehicle, accommodate customers, assess injuries, identify spill, spread absorbent, etc.
E4_5	Investigating and Documenting Collisions and Other Incidents	Fill in the blank	6	When evaluating a collision/incident, interview _____ first.

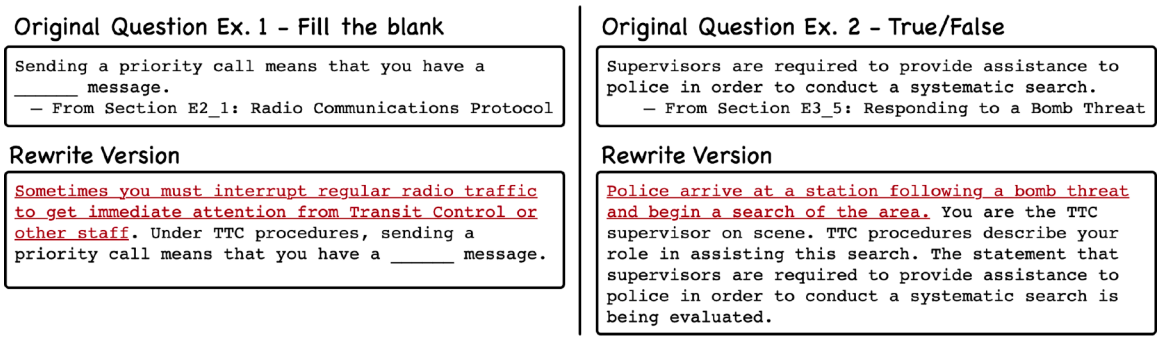


Fig. 8. Examples of original training quiz items (top) and rewritten versions (bottom). The rewritten questions inject short narrative scenarios and more natural phrasing while preserving the underlying handbook grounding, making them closer to real incident descriptions posed by on-road supervisors.

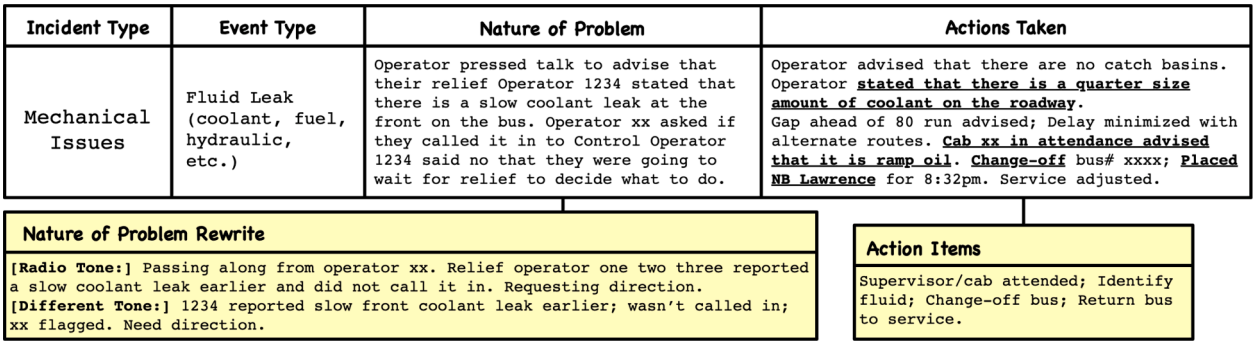


Fig. 9. Example route 29 incident record used for the knowledge graph. The top row shows the original structured fields (*incident type*, *event type*, *nature of problem*, *actions taken*). The highlighted boxes illustrate (bottom left) rewritten *nature of problem* in two radio-styles narrative form and (bottom right) the action keywords extracted from *actions taken*, which are used as reference actions when evaluating ELSSA's generated suggestions.

the knowledge base and 37 records for evaluation in each fold. All reported metrics are averaged over the five folds (mean ± standard deviation).

We evaluate suggestion quality by comparing generated actions with those recorded by TTC staff. For each test record, we manually extract a small set of action keywords from the *Actions Taken* field (Fig. 9). To mimic different user communication styles, we also reformat the original *Nature of Problem* descriptions into two additional variants: (i) a radio-communication tone that resembles short, telegraphic supervisor–control exchanges, and (ii) an alternative narrative tone that paraphrases the same information in more conversational language. An example of the original record, the rewritten problem description, and the extracted action items is shown in Fig. 9. These variants allow us to test the robustness of the retrieval and reasoning pipeline to variations in how supervisors describe incidents.

The cross-validation above measures performance within a single route and time period, which leaves open whether the system generalizes to genuinely new reports. To probe this, we assemble a second evaluation set **(*2023 DB*)** with 87 incident reports, which are drawn from a different and earlier window SOR from May–November 2023, while the knowledge base remains the historical Route 29 collection. One thing we should address is that for the 2023 DB, it only covers incident reports under the categories of investigation (74) and cleaning (13). It acts as a deployment proxy: it mirrors the operational reality that the system is queried with freshly received incidents whose resolutions it has never seen, and must instead reason from accumulated experience.

4.1.1. Evaluation metrics

We evaluate the performance of ELSSA on experience-based suggestion along *decision quality* and the *operational cost*.

Decision quality: action-code metrics. We evaluate decision quality at the level of operational actions rather than surface text similarity. This distinction is important because a recommendation may be linguistically similar to a reference narrative while prescribing an incomplete, unsupported, or operationally inappropriate response. To capture this dimension, both the model-generated recommendation and the human-written *Actions Taken* field are mapped to a controlled vocabulary of canonical action codes spanning supervisor-relevant response categories (Appendix F). Let P denote the set of action codes predicted from ELSSA's recommendation, G the reference set extracted from the supervisor-documented *Actions Taken* field, and E the set of action codes that can be derived from the retrieved evidence $\mathcal{R}$. The reference set G therefore represents the actions that were actually taken and recorded in the historical incident report, while E represents the actions that are explicitly supported by retrieved precedents.

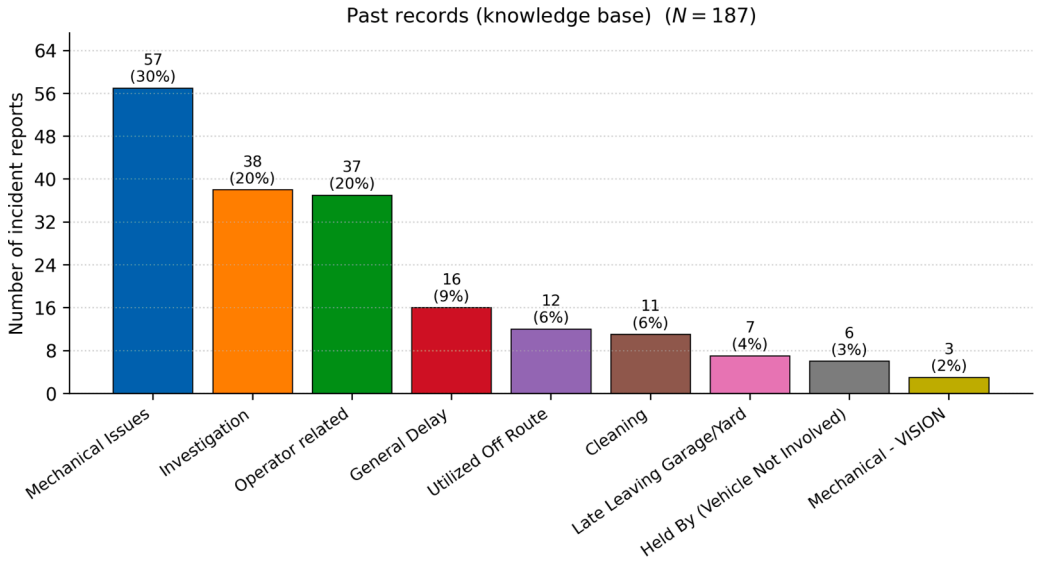


Fig. 10. Distribution of route 29 incident reports by incident type (November 2024, $N = 187$). Mechanical issues, investigation, and operator-related incidents account for the largest shares of the dataset.

We report precision, recall, and F_1 score as follows:

$$\text{Precision} = \frac{|P \cap G|}{|P|}, \qquad \text{Recall} = \frac{|P \cap G|}{|G|}, \qquad F_1 = \frac{2\,\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Precision measures the extent to which the recommended actions correspond to actions actually taken by supervisors, whereas recall measures the extent to which the recommendation recovers the documented operational response. We use F_1 as the primary decision-quality summary because it jointly penalizes over-prescription and omission.

In addition, we report an evidence-based fabrication rate:

$$\text{Fabrication rate} = \frac{|P \setminus E|}{|P|}.$$

Unlike precision and recall, which compare the recommendation against the recorded supervisory response, the fabrication rate compares the recommendation against the retrieved evidence. It measures the share of recommended action codes that are not supported by any retrieved precedent. We use this metric as the main operational proxy for over-prescription or hallucination, with lower values indicating better evidence alignment. The use of canonical action codes makes this evaluation robust to paraphrase: recommendations that differ lexically but prescribe the same operational action are mapped to the same code, addressing a key limitation of surface-overlap metrics.

Structural safety and deployment-cost metrics. We report inference latency using the mean and the $p50$, $p95$, and $p99$ wall-clock time per query. This is included because deployment in a control room depends not only on average performance but also on tail latency. A system that is accurate on average but occasionally produces delayed responses may be unsuitable for time-sensitive supervisory decision support.

LLM-based action quality judgment. To complement the action-code metrics, we used an LLM-as-judge protocol to assess the operational quality of each generated recommendation against the historically recorded actions. The judge evaluated each candidate response along five rubric dimensions. *Action coverage* measures whether the recommendation captures the main control actions documented in the historical record. *Resource alignment* assesses whether the recommendation calls for the same external or operational resources, such as towing, supervision, emergency response, or other field support. *Impact handling* evaluates whether the response addresses the relevant service impacts, such as gaps, out-of-service movements, delays, or passenger disruption. *Safety* captures whether the recommendation avoids unsafe or operationally inappropriate actions. *Groundedness* measures whether the recommendation is supported by the retrieved evidence rather than introducing unsupported actions or assumptions. Each dimension was scored on an integer scale from 1 to 5, where higher values indicate stronger agreement with the historical actions and retrieved evidence. The overall judge score is computed as the unweighted average of the five rubric scores. The exact judge prompt is reproduced in Appendix B.

4.2. Experimental environment

Our experiments are conducted under the following setup:

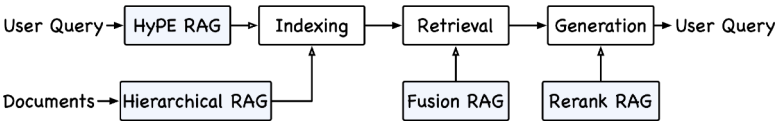


Fig. 11. Baseline RAG configurations used for comparison. For indexing phase, HyPE RAG operates on the query side, Hierarchical RAG structures the document index; then Fusion RAG augments the retrieval stage, and Rerank RAG refines the final context passed to the generator. Together, they probe improvements at different stages of the RAG pipeline.

- **Programming language:** Python 3.12.
- **Vector database:** FAISS for storing document and entity embeddings.
- **Inference LLM:** GPT-4o-mini as the primary back-end model.
- **Evaluation LLM:** GPT-5-mini for automatic assessment.
- **Embedding model:** `text-embedding-3-small` for all queries, SOP chunks, and KG node descriptions (unless otherwise noted).
- **RAG and agent frameworks:** LangChain for RAG pipelines; LangGraph and Microsoft GraphRAG for agent orchestration.
- **User interface:** Streamlit-based interactive supervisor-facing application.

4.3. RAG system performance for rule-based and experience-based incident control suggestions

4.3.1. Benchmark RAG systems

To benchmark the proposed RAG architectures, we implement several stronger baselines that represent different design choices in the RAG pipeline:

- **Naive RAG.** A basic chunk-level RAG system that encodes all document content into a vector store and retrieves the top-k most similar chunks by cosine similarity, which are then appended to the LLM prompt.
- **Reranking RAG** (Mortaheb et al., 2025). An extension of naive RAG that first performs standard dense retrieval to obtain an initial candidate set, then forms query–document pairs and applies either (i) an LLM-based scoring prompt or (ii) a cross-encoder to compute relevance scores. Documents are reordered by these scores, and only the top-k reranked chunks are passed to the generator.
- **Fusion RAG.** A hybrid retrieval baseline that combines dense (vector-based) retrieval with sparse keyword matching (e.g., BM25). Scores from the two retrieval channels are fused (e.g., via weighted combination or reciprocal rank fusion) to improve robustness across both keyword-heavy and semantically phrased queries.
- **HyPE RAG** (Vake et al., 2025). A RAG system enhanced with Hypothetical Prompt Embeddings (HyPE). During indexing, hypothetical questions are pre-computed for each document segment, and their embeddings are stored alongside the original content. At query time, retrieval is cast as question–question matching between the user query and these hypothetical questions, mitigating style mismatch between training documents and user queries.
- **Hierarchical RAG** (Liang et al., 2025). A hierarchical indexing baseline with two levels of encoding: document- or section-level summaries and fine-grained chunks. Retrieval proceeds in two stages: first, summaries are used to select a small set of relevant sections; second, detailed chunks are retrieved only within those sections, improving both efficiency and relevance.

As summarized in Fig. 11, these baselines collectively cover major components of the RAG pipeline—retrieval strategy, reranking, dense–sparse fusion, style-robust retrieval, and hierarchical indexing—providing a broad comparison point for ELSSA's agentic, hierarchy-aware and knowledge-graph-based RAG modules.

4.3.2. Model performance on rule-based incident control suggestions

Table 2 reports four metrics for each RAG variant on both the original and rewritten quiz sets: *First-hit* (fraction of questions for which the correct handbook section is ranked first), *Hit* (fraction for which the correct section appears anywhere in the retrieved set), answer *Accuracy*, and the *Gap* |Hit − Accuracy| between retrieval quality and answer quality. Overall, ELSSA clearly outperforms all baselines. On the original questions, it matches the best-performing baseline (rerank RAG) in accuracy (0.821) while achieving a higher first-hit rate (0.714 vs. 0.696). On the rewritten questions, which more closely resemble realistic supervisor queries, ELSSA attains the highest scores on every metric (first-hit = 0.875, hit = 0.946, accuracy = 0.929) and the smallest gap (0.018). This indicates that ELSSA not only retrieves the appropriate SOP sections more reliably, but also converts that evidence into correct answers with minimal loss, and that its advantage grows as queries move from template-style items to naturalistic descriptions.

The baselines reveal additional patterns. Simple (naive) RAG performs surprisingly well on the original quiz, reflecting that many questions are stylistically very close to the handbook text (often near-verbatim sentences), where straightforward semantic similarity search already works well. Hierarchical RAG, despite relatively high hit rates, exhibits the largest gaps (0.286 and 0.131), suggesting that structural indexing alone is insufficient: the correct section is often retrieved, but the downstream answer is still wrong. HyPE RAG underperforms on all metrics and degrades on the rewritten quiz, consistent with the idea that hypothetical question generation is unnecessary—and potentially harmful—when queries already match the SOP style. Fusion RAG and rerank RAG improve over naive RAG by strengthening retrieval, yet ELSSA surpasses them, especially on the rewritten quiz, highlighting the benefit of agentic, hierarchy-aware selection of a small set of relevant sections over document-scale retrieval with purely score-based fusion or reranking.

Table 2
Performance of RAG variants on the supervisor quiz under the original and rewritten question sets. "Gap" is the absolute difference between retrieval hit rate and answer accuracy (smaller is better). ELSSA is our agentic, hierarchy-aware RAG system.

	Original quiz				Rewritten quiz			
Model	First-hit	Hit	Accuracy	Gap	First-hit	Hit	Accuracy	Gap
Simple RAG	0.625	0.804	0.714	0.089	0.714	0.875	0.821	0.054
Fusion RAG	0.196	0.589	0.500	0.089	0.286	0.661	0.643	0.018
Hierarchical RAG	0.286	0.750	0.464	0.286	0.250	0.821	0.691	0.131
HyPE RAG	0.250	0.554	0.446	0.107	0.143	0.357	0.411	0.054
Rerank RAG	0.696	0.875	0.821	0.054	0.679	0.929	0.839	0.089
ELSSA	**0.714**	**0.875**	**0.821**	**0.054**	**0.875**	**0.946**	**0.929**	**0.018**

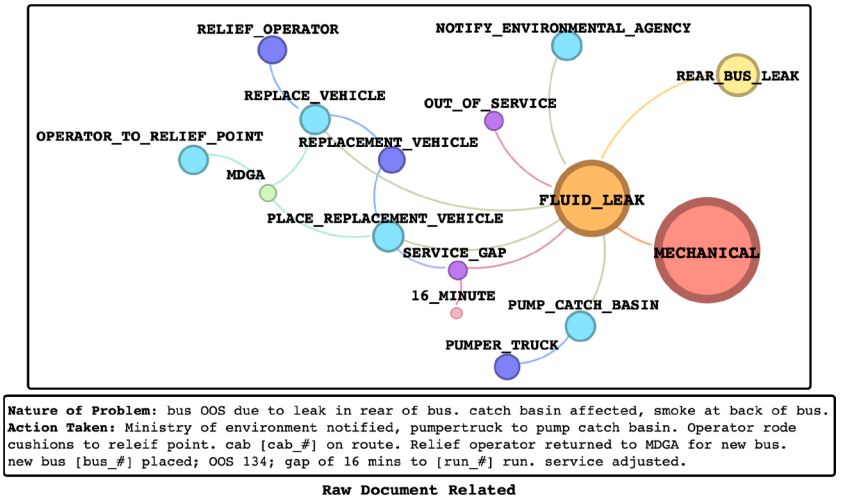


Fig. 12. Example neighbourhood of the incident knowledge graph for a mechanical FLUID_LEAK event (left), with the corresponding raw incident description (right). The graph links incident pattern, event category, service impacts, action strategies, resources, locations, and measurements, providing a structured summary of the actions taken in response to the incident.

However, the efficiency gains from ELSSA should be interpreted in terms of end-to-end decision quality rather than the number of LLM calls alone. On average, ELSSA makes 5.607 LLM calls per query, reflecting its iterative agent-in-the-loop retrieval process, including LLM-based retrieval stopping and final recommendation generation. This is higher than simpler agent-in-workflow baselines such as Rerank RAG, which requires only two LLM calls during the retrieval and synthesis process.

4.3.3. Performance on experience-based incident control suggestions

Quality of the extracted incident knowledge graph. We first examine the structure and consistency of the incident knowledge graph that underpins experience-based retrieval. Using the LLM-guided extraction procedure described in Section 3, we obtain a graph with 1,053 distinct entities and 1,947 relations. Fig. 12 illustrates a local neighbourhood for a FLUID_LEAK event (detailed entities and relationships extracted are present in Appendix D). The subgraph links a mechanical incident pattern to its event category, service impacts, action strategies, resource requirements, location patterns, and associated time measurements. The corresponding raw report (bottom of Fig. 12) shows that these nodes collectively summarize the operational storyline: environmental agency notification, pumping the catch basin, arranging a replacement bus, and restoring service after a quantified gap. This example illustrates that the extracted graph captures the causal chain from incident pattern to service impact and resolution strategy in a compact, machine-navigable form.

Table 3 summarizes the entity-type distribution before and after applying the auto-regularization procedure. The initial graph contains 106 entities labelled as UNKNOW, reflecting cases where the LLM could not confidently assign a type. After regularization, this category disappears ($\Delta = -106$), and the counts of semantically meaningful types increase, while INCIDENT_PATTERN remains unchanged. This pattern indicates that the proposed auto-regularization algorithm successfully reclassifies previously untyped entities into the operational categories most consistent with their observed relation neighbourhoods, rather than creating new spurious types.

Fig. 13 quantifies how regularization affects relation semantics. For each relation type, we report the fraction of relation instances accounted for by the two most frequent *(source-type, target-type)* pairs before and after regularization. The top-2 coverage improves across all relations, often substantially: for example, for *HAPPENS_AT* it rises from 0.70 to 0.91, for *REQUIRES* from 0.83 to 0.97, and for *RESULTS_IN* from 0.89 to 0.99. At the pair level, edges that violate our schema—such as reversed directions (e.g., RESOURCE_REQUIREMENT → ACTION_STRATEGY instead of ACTION_STRATEGY → RESOURCE_REQUIREMENT) or ill-typed combinations in-

Table 3
Entity-type counts and changes after applying the auto-regularization algorithm. The Δ column reports the change in the number of entities of each type (Positive values indicate an increase, negative values a decrease). *UNKNOW denotes entities that were initially untyped.

Entity type	Count	Δ after auto-regularization
ACTION_STRATEGY	298	+25
ROOT_CAUSE	169	+24
MEASUREMENT	159	+15
LOCATION_PATTERN	144	+17
UNKNOW*	106	−106
RESOURCE_REQUIREMENT	64	+14
EVENT_CATEGORY	56	+2
SERVICE_IMPACT	48	+9
INCIDENT_PATTERN	9	0

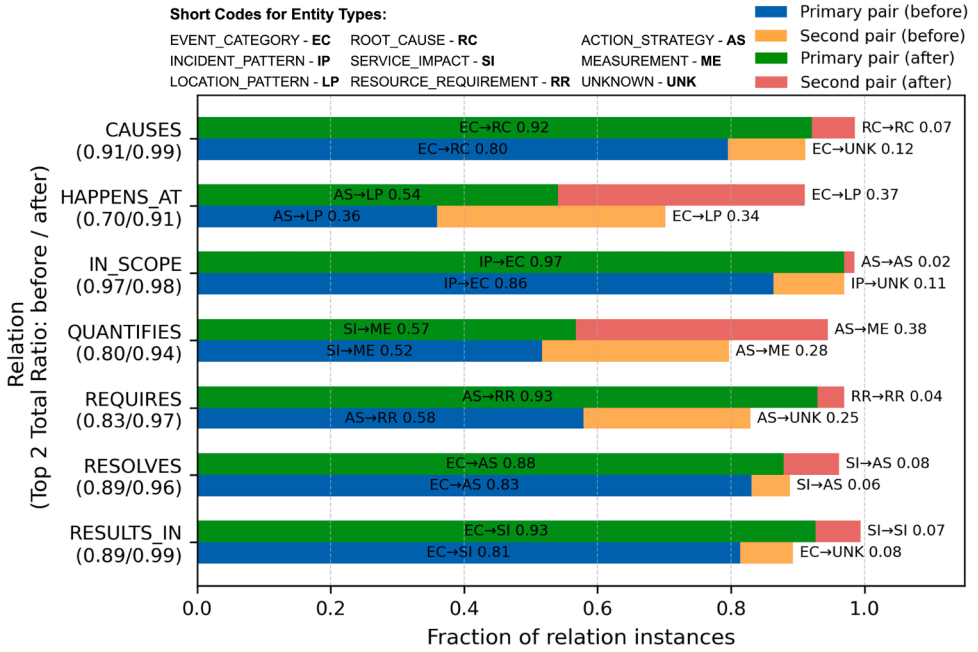


Fig. 13. Effect of auto-regularization on relation type signatures. For each relation, bars show the fraction of instances accounted for by the two most frequent *(source-type, target-type)* pairs before (blue/orange) and after (green/red) regularization. Numbers in parentheses indicate the combined share of the top two pairs before/after. The procedure strengthens canonical type pairs (e.g., `EVENT_CATEGORY` → `ROOT_CAUSE` for *CAUSES*, `ACTION_STRATEGY` → `RESOURCE_REQUIREMENT` for *REQUIRES*) and suppresses inconsistent or unknown-type edges.

volving `UNKNOW`—are either corrected or removed, yielding a graph that is both more interpretable and more homogeneous. Importantly, these improvements are achieved using only lightweight frequency statistics, with negligible computational overhead.

Overall, the auto-regularization step substantially increases the semantic coherence of the knowledge graph. It enforces directionality consistent with the operational schema, fills in missing entity types, and removes residual noise. This yields a graph that is better suited for multi-hop traversal and for interpreting retrieved paths as meaningful incident narratives rather than artifacts of extraction noise.

Evaluation of experience-based incident control suggestions. We next evaluate whether the proposed knowledge-graph-based RAG module can support experience-based incident control suggestions. For each test incident, the system retrieves relevant historical entities, relations, and graph-derived contextual summaries, and then generates a recommended set of control actions. The evaluation is conducted from two complementary perspectives. First, we use action-code metrics to assess whether the generated recommendation contains the operational actions documented in the historical *Actions Taken* field. Specifically, both the generated recommendation and the historical action record are mapped to a controlled set of canonical action codes, from which we compute code-level F_1, recall, precision, and fabrication rate. Code recall measures whether historically taken actions are recovered, code precision measures whether recommended actions are consistent with the historical record, and code fabrication captures the share of recommended action codes that are not supported by the historical action set.

Table 4
Accuracy of RAG variants on experience-based incident control suggestions across input tones and the recent held-out database. "AR" denotes the auto-regularization algorithm applied to the knowledge graph.

System	Code F_1	Code Recall	Code Prec.	Code Fab. ↓
Original tone (5-fold CV)				
Simple RAG	0.650 ± 0.039	0.709 ± 0.043	0.648 ± 0.040	0.201 ± 0.021
Fusion RAG	0.654 ± 0.031	0.705 ± 0.031	0.665 ± 0.047	0.183 ± 0.056
HyPE RAG	0.657 ± 0.042	0.713 ± 0.039	0.662 ± 0.040	0.180 ± 0.037
Rerank RAG	0.654 ± 0.027	0.690 ± 0.038	0.677 ± 0.033	0.168 ± 0.047
Hierarchical RAG	0.622 ± 0.042	0.689 ± 0.036	0.620 ± 0.038	0.213 ± 0.056
ELSSA	**0.733 ± 0.029**	0.819 ± 0.016	**0.715 ± 0.043**	0.111 ± 0.036
ELSSA w/ AR	0.732 ± 0.029	**0.829 ± 0.015**	0.707 ± 0.041	**0.107 ± 0.015**
Radio-communication tone (5-fold CV)				
Simple RAG	0.608 ± 0.044	0.684 ± 0.043	0.602 ± 0.054	0.237 ± 0.043
Fusion RAG	0.626 ± 0.027	0.682 ± 0.029	0.626 ± 0.045	0.220 ± 0.046
HyPE RAG	0.596 ± 0.045	0.669 ± 0.038	0.591 ± 0.049	0.236 ± 0.062
Rerank RAG	0.613 ± 0.038	0.679 ± 0.057	0.615 ± 0.048	0.226 ± 0.042
Hierarchical RAG	0.610 ± 0.038	0.681 ± 0.046	0.607 ± 0.033	0.239 ± 0.022
ELSSA	**0.688 ± 0.054**	**0.801 ± 0.035**	**0.662 ± 0.064**	0.156 ± 0.042
ELSSA w/ AR	0.676 ± 0.060	0.797 ± 0.038	0.640 ± 0.074	**0.154 ± 0.037**
Paraphrased narrative tone (5-fold CV)				
Simple RAG	0.641 ± 0.053	0.730 ± 0.049	0.626 ± 0.064	0.190 ± 0.036
Fusion RAG	0.621 ± 0.056	0.680 ± 0.034	0.629 ± 0.074	0.185 ± 0.022
HyPE RAG	0.587 ± 0.042	0.655 ± 0.034	0.585 ± 0.047	0.237 ± 0.035
Rerank RAG	0.620 ± 0.060	0.671 ± 0.065	0.625 ± 0.068	0.186 ± 0.038
Hierarchical RAG	0.670 ± 0.028	0.744 ± 0.022	0.653 ± 0.039	0.191 ± 0.015
ELSSA	0.681 ± 0.032	**0.807 ± 0.012**	0.647 ± 0.044	0.143 ± 0.043
ELSSA w/ AR	**0.686 ± 0.051**	0.792 ± 0.034	**0.661 ± 0.053**	**0.111 ± 0.029**
Recent database (May–No. 2023, held-out; original tone)				
Simple RAG	0.564	0.579	0.621	0.240
Fusion RAG	0.601	0.647	0.648	0.229
HyPE RAG	0.630	0.653	**0.670**	0.205
Rerank RAG	0.574	0.636	0.600	0.239
Hierarchical RAG	0.572	0.592	0.634	0.236
ELSSA	0.628	0.726	0.648	0.176
ELSSA w/ AR	**0.655**	**0.738**	0.668	**0.157**

Second, because operational decision quality cannot be fully characterized by discrete action-code overlap alone, we also report an LLM-as-judge evaluation in Table 5. The judge scores each recommendation along five dimensions: action coverage, resource alignment, impact handling, safety, and groundedness as defined in Section 4.1.1. The overall judge score is computed as the average of the five rubric scores. The exact judge prompt is reported in Appendix B.

Tables 4 and 5 report performance under three input styles: the original occurrence-report wording, a concise radio-communication style, and a paraphrased narrative style. These variants test whether the retrieval and reasoning pipeline remains stable when the same operational incident is described using different natural-language forms. In addition, the 2023 DB provides a temporally separated held-out evaluation set, which evaluates whether the knowledge base built from historical Route 29 experience can support recommendations for newly received reports whose resolutions are not available to the system at inference time.

The results show that ELSSA consistently improves decision-quality metrics over conventional RAG baselines. Under the original tone, ELSSA achieves the highest code F_1 score of 0.733, compared with 0.650–0.657 for other RAG structure. The improvement is driven by both higher recall and higher precision: ELSSA recovers a substantially larger share of historically documented actions while also maintaining the strongest code precision. This indicates that the graph-structured representation does not merely retrieve more evidence; it retrieves more operationally relevant evidence. In contrast, the baseline RAG systems remain closer to lexical or embedding-level similarity and are therefore less able to recover action patterns that depend on structured relationships among incident type, event type, problem description, and historically observed response strategies.

The advantage of ELSSA becomes more clear when input language deviates from the original report style. Under the radio-communication tone, ELSSA achieves the best code F_1 of 0.688 and the highest recall of 0.801, outperforming all baseline RAG variants. Under the paraphrased narrative tone, ELSSA w/ AR achieves the best code F_1 of 0.686, the best precision of 0.661, and the lowest fabrication rate of 0.111. This pattern is important because practical control-room queries are unlikely to reproduce the wording of historical reports exactly. Supervisors may describe the same event in shorthand, in radio-style fragments, or in more conversational language. The fact that ELSSA and ELSSA w/ AR remain competitive under these rewritten inputs suggests that the knowledge graph helps shift retrieval away from surface-level textual matching and toward more stable operational structure.

Table 5
Per-rubric LLM-as-judge scores for RAG variants on experience-based incident control suggestions across input tones and the recent held-out database. "AR" denotes the auto-regularization algorithm applied to the knowledge graph.

System	Action cov.	Resource align.	Impact hand.	Safety	Grounded.	Overall
Original tone (5-fold CV)						
Simple RAG	4.19	2.81	3.72	4.95	4.57	4.05
Fusion RAG	4.13	2.82	3.69	**4.98**	4.50	4.02
HyPE RAG	4.03	2.65	3.62	4.92	4.45	3.93
Rerank RAG	4.15	2.91	3.75	**4.98**	4.48	4.05
Hierarchical RAG	4.27	**2.97**	3.79	4.96	**4.59**	4.12
ELSSA	4.20	2.84	3.80	4.97	4.55	4.07
ELSSA w/ AR	**4.35**	2.90	**3.90**	4.97	4.58	**4.14**
Radio-communication tone (5-fold CV)						
Simple RAG	4.20	2.95	3.84	4.95	4.52	4.09
Fusion RAG	4.18	2.92	3.85	4.93	4.55	4.09
HyPE RAG	4.14	2.83	3.76	4.95	4.50	4.04
Rerank RAG	4.19	2.89	3.77	4.96	4.55	4.07
Hierarchical RAG	4.27	2.97	3.84	4.94	4.55	4.11
ELSSA	4.32	2.99	3.89	**4.98**	4.61	4.16
ELSSA w/ AR	**4.33**	**3.06**	**3.97**	4.97	**4.63**	**4.19**
Paraphrased narrative tone (5-fold CV)						
Simple RAG	4.26	2.92	3.88	4.95	4.55	4.11
Fusion RAG	4.22	2.91	3.78	4.96	4.52	4.08
HyPE RAG	4.04	2.74	3.78	4.94	4.50	4.00
Rerank RAG	4.22	2.97	3.82	4.91	4.55	4.09
Hierarchical RAG	4.28	2.99	3.88	4.96	4.56	4.13
ELSSA	4.38	3.01	**4.00**	**4.98**	4.65	4.20
ELSSA w/ AR	**4.45**	**3.02**	**4.00**	**4.98**	**4.70**	**4.23**
Recent database (May–No. 2023, held-out; original tone)						
Simple RAG	4.06	3.17	3.59	4.95	4.44	4.04
Fusion RAG	4.05	2.97	3.71	**4.99**	4.45	4.03
HyPE RAG	4.01	2.80	3.53	4.90	4.40	3.93
Rerank RAG	4.21	3.11	3.77	4.91	4.51	4.10
Hierarchical RAG	4.08	3.25	3.69	4.97	4.47	4.09
ELSSA	**4.29**	3.25	**4.03**	4.98	4.48	4.21
ELSSA w/ AR	**4.29**	**3.30**	4.02	4.98	**4.54**	**4.23**

This finding is also consistent with prior work on the limitations of LLM-based natural language understanding. Zhong et al. show that ChatGPT-style models can perform strongly on inference-oriented tasks but are weaker on paraphrase and semantic similarity tasks compared with fine-tuned BERT-style models on GLUE benchmarks. Zhong et al. (2023) This is directly relevant to the present setting: experience retrieval for incident control is partly a paraphrase and similarity problem, because the system must recognize that two differently worded problem descriptions may imply the same operational condition. The performance gap between conventional RAG and ELSSA under the radio and paraphrased tones suggests that graph-structured retrieval can partially compensate for this known weakness by grounding the similarity search in typed entities, relations, and community-level operational context rather than relying only on the LLM's implicit language understanding or dense textual similarity.

Meanwhile, we observed that the proposed Auto-regularization further improves the reliability of the graph-based retrieval process. ELSSA w/ AR applies rule-based regularization to improve the consistency of entity and relation extraction in the knowledge graph. Its main benefit is not simply a uniform increase across all metrics, but a more deployment-relevant trade-off: it reduces unsupported action fabrication while preserving or improving action recovery. Across the three 5-fold evaluation panels, ELSSA w/ AR obtains the lowest fabrication rate in every tone. This reduction is operationally meaningful. In an incident-control setting, an unsupported recommendation can be more harmful than an omitted recommendation, because it may direct staff toward unnecessary resources, or actions that are not justified by the incident evidence. Lower fabrication therefore provides a practical proxy for reduced hallucination risk.

The temporally separated 2023 DB further supports the robustness of ELSSA w/ AR. Compared with ELSSA without auto-regularization, ELSSA w/ AR improves F_1 from 0.628 to 0.655 and reduces fabrication from 0.176 to 0.157. The improvement suggests that a more regularized graph representation is less sensitive to incidental extraction noise in the historical knowledge base and better able to retrieve transferable operational experience for newly received incidents.

The LLM-as-judge results in Table 5 provide a broader view of response quality. ELSSA w/ AR achieves the highest overall judge score in every evaluation panel. The largest differences appear in action coverage, resource alignment, impact handling, and groundedness, which correspond closely to the practical requirements of a transit control recommendation. The safety scores are uniformly high across methods, indicating that most systems avoid clearly unsafe recommendations. However, high safety alone is

Table 6
Per-query latency of RAG variants under 5-fold cross-validation, reported in seconds as mean ± standard deviation across folds. Lower values are better. "AR" denotes the auto-regularization algorithm applied to the knowledge graph.

System	$p50$	$p95$	Mean
Simple RAG	2.781 ± 0.086	4.384 ± 1.000	2.809 ± 0.109
Fusion RAG	**2.671 ± 0.124**	3.889 ± 0.487	**2.695 ± 0.091**
HyPE RAG	3.707 ± 0.307	5.218 ± 0.326	3.733 ± 0.368
Rerank RAG	2.762 ± 0.201	**3.805 ± 0.173**	2.706 ± 0.111
Hierarchical RAG	4.257 ± 0.702	7.506 ± 0.643	4.513 ± 0.537
ELSSA	3.720 ± 0.164	5.464 ± 0.637	3.824 ± 0.162
ELSSA w/ AR	2.964 ± 0.108	6.687 ± 2.908	2.777 ± 0.504

Rare LLM-API stalls longer than 20 s were excluded.

insufficient for operational usefulness; a safe but generic response may still fail to recover the specific action sequence, resource request, or service-impact mitigation strategy used historically. The combined action-code and rubric-level results therefore show that ELSSA w/ AR improves both discrete action recovery and qualitative operational completeness.

Table 6 reports per-query latency for the experience-based recommendation systems. The results show that the proposed approach remains within a practical response-time range for decision support. Conventional RAG baselines are fastest, with Fusion RAG and Rerank RAG averaging 2.695 and 2.706 s per query, respectively. ELSSA incurs additional retrieval overhead because it performs graph-based evidence construction before synthesis, but its mean latency remains moderate at 3.824 s. This overhead is acceptable for incident-control decision support, where the system is intended to assist human supervisors rather than directly actuate control decisions at sub-second frequency.

Interestingly, ELSSA w/ AR is more efficient than the unregularized ELSSA run, with a mean latency of 2.777 s. Because the auto-regularization procedure is applied to improve the quality and consistency of the knowledge graph before online querying, it does not introduce substantial additional inference-time cost. Instead, a cleaner graph can reduce retrieval ambiguity and lead to a more compact and consistent evidence context. In deployment terms, this is an important property: auto-regularization improves robustness and reduces fabrication while preserving real-time usability.

4.3.4. SOP compliance results

After filtering both datasets to *Investigation* incidents, cleaning, and de-duplication, the evaluation set contains **112 incidents** (38 from the previously cleaned collection and 74 from the temporally separated 2023 test set). Across 10 repeated runs, ELSSA generated on average **842 SOP recommendations** (≈7.5 per incident).

Compliance by operational category. Over the source-supported, applicable recommendations, the compliance rate—the share explicitly documented as followed in the SOR narrative—is **79.2**% ± **0.5**%. The across-run standard deviations are small (well under one percentage point), indicating that, with retrieval held fixed, the end-to-end pipeline is stable despite its stochastic LLM components (per-run strict rates lie within $[78.6\%, 80.1\%]$). Re-running the comparison with an independent judge (`gpt-4o-mini`) yields a closely matching strict rate of $78.9\% \pm 0.4\%$; the two judges agree at the item level with Cohen's $\kappa = 0.70 \pm 0.01$, and the primary judge's repeated-run self-consistency is Fleiss' $\kappa = 0.80$ (label agreement 91.6% in all runs).

Fig. 14 summarizes SOP compliance at the operational-category level after applying the revised declarative crediting rules, including report-intrinsic credits and medical-response equivalences. A pronounced documentation gradient emerges.

- **Service Operations** and **Evidence & Documentation** are the most consistently documented categories. Service Operations reaches $95.5\% \pm 0.9\%$ compliance, with an average denominator of approximately 560 applicable code instances per run. Evidence & Documentation reaches $94.0\% \pm 1.6\%$, with approximately 127 applicable instances per run. This pattern is expected because SORs are structured primarily around service accountability: what happened, how service was affected, how long the incident lasted, and how normal operation was restored. Core report fields such as time, location, vehicle identifiers, employee identifiers, and injury status are also routinely recorded.
- **Communication & Investigation** shows intermediate compliance at $77.2\% \pm 0.6\%$, with approximately 366 applicable instances per run. This category contains two distinct types of actions. Report-intrinsic activities, such as notifying Transit Control, updating the situation, and conducting a basic investigation, are usually documented or inferable from the SOR itself. By contrast, conditional investigative steps, especially interviewing involved parties or witnesses and preserving video or other evidence, are much less consistently mentioned.
- **Scene Management**, **Safety & Medical Response**, and **Support & Aftercare** are the least visible categories in the SOR narratives. Scene Management reaches only $38.5\% \pm 2.1\%$, Safety & Medical Response reaches $20.0\% \pm 4.8\%$ under strict crediting, and Support & Aftercare reaches $15.4\% \pm 3.1\%$. These categories consist largely of conditional on-scene actions, such as securing the scene, managing passengers, using PPE, assessing injuries, offering employee assistance, or documenting stress-related removal from duty. Their low compliance should *not* be interpreted as evidence that the actions were not performed. Rather, it indicates that brief SOR narratives often *prioritize service recovery and reportable outcomes over procedural scene-safety and aftercare details*.

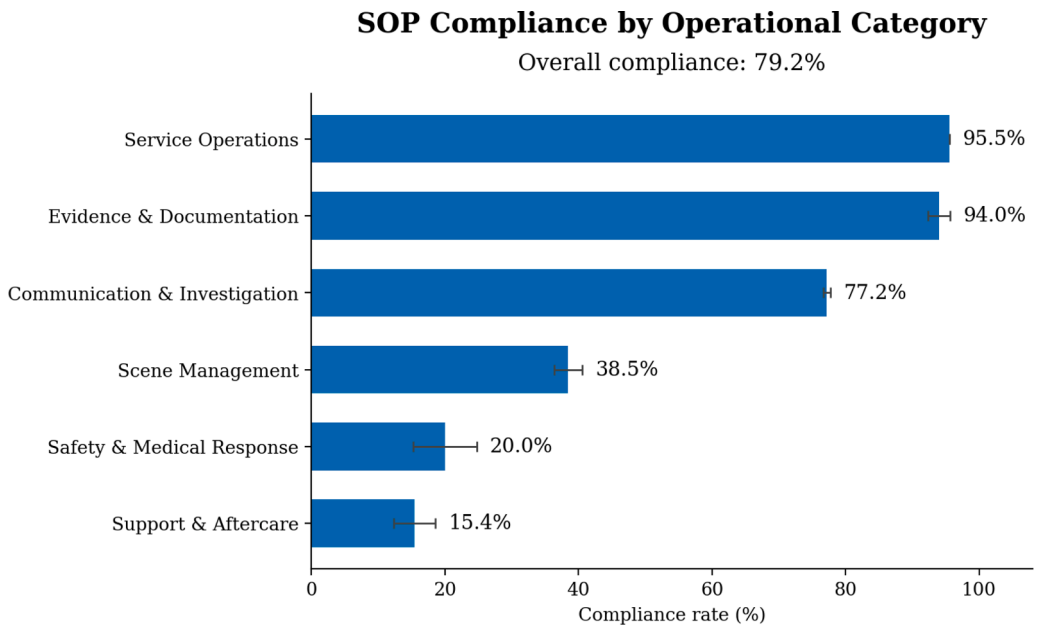


Fig. 14. SOP compliance by operational category.

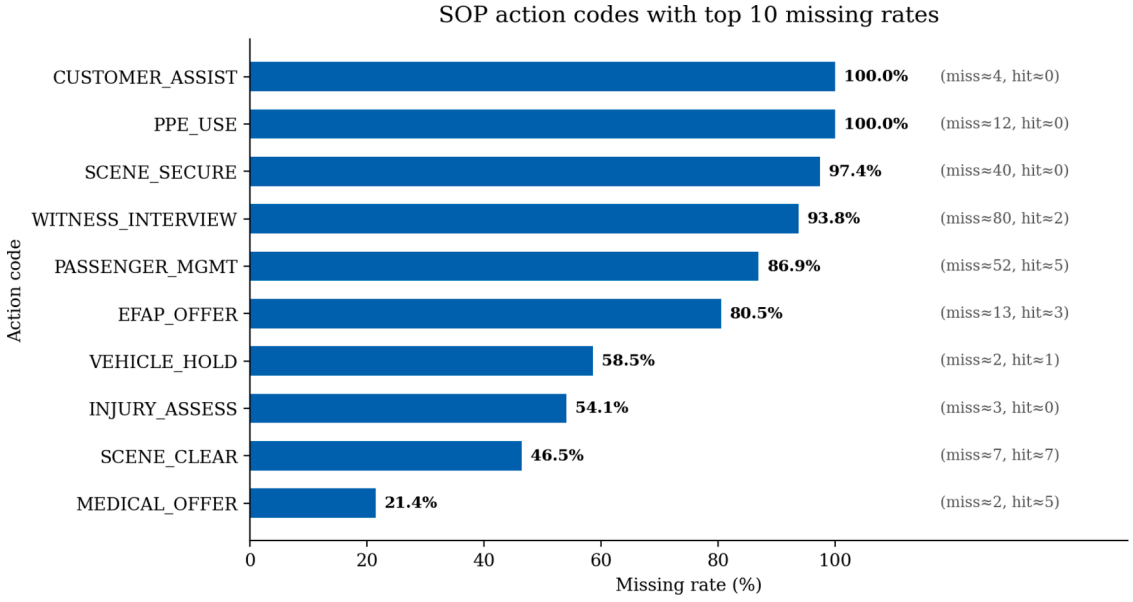


Fig. 15. Top ten SOP action codes ranked by missing rate. For each code, the percentage indicates how often ELSSA recommended the action but the corresponding SOR did not document it. The text on the right reports the underlying counts (miss, hit).

Frequently missing action codes. To better understand where gaps arise, Fig. 15 lists the ten action codes with the highest missing rates. Several distinct patterns appear.

- **Highly missing on-scene and aftercare actions.** The highest missing rates are concentrated in actions that are conditional, situational, and often peripheral to the main service-restoration narrative. `CUSTOMER_ASSIST` and `PPE_USE` both show 100% missing rates, while `SCENE_SECURE` is missing in 97.4% of applicable instances. These codes correspond to actions such as providing non-medical customer assistance, using protective equipment, or explicitly securing the incident scene. Their absence from the SOR should not be interpreted as definitive evidence that the actions were not performed. Rather, these results indicate that such procedural details are rarely mentioned unless they are central to the incident outcome.
- **Investigation and passenger-management details are frequently under-mentioned.** `WITNESS_INTERVIEW` has a very high missing rate of 93.8%, with approximately 80 missing instances and only 2 documented hits. Similarly, `PASSENGER_MGMT` is missing

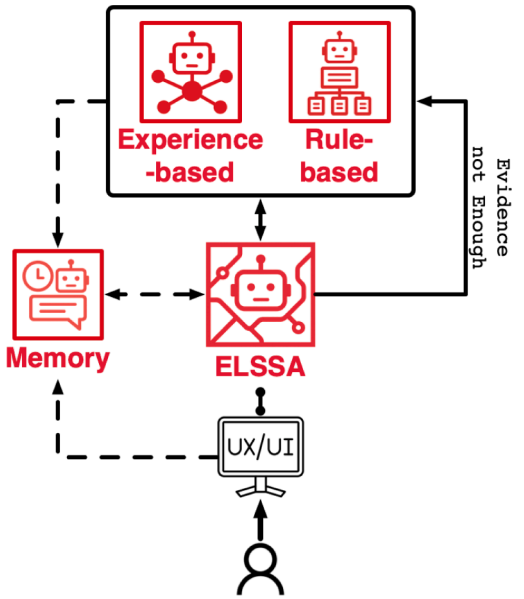


Fig. 16. High-level ELSSA architecture. A central LLM agent orchestrates rule-based SOP retrieval and experience-based retrieval over the incident knowledge graph, fuses the evidence into a response, and stores key context in a short-term memory buffer to support follow-up queries. When retrieved evidence is insufficient, ELSSA explicitly signals this and requests clarification or additional details from the user.

in 86.9% of applicable cases. These results suggest that the SOR often records the incident outcome but not the full procedural path by which the supervisor established facts or managed affected passengers. For example, supervisors may speak with operators, customers, or witnesses during an incident, but the final SOR may summarize only the resulting finding or disposition. Likewise, passenger offloading, transfer, or assistance may be treated as routine operational handling and omitted from the narrative.

- **Support and employee-care actions are sparsely visible.** `EFAP_OFFER` is missing in 80.5% of applicable instances. This is particularly important because employee support is a meaningful aftercare action but is only visible when the SOR explicitly states that EFAP or equivalent assistance was offered, accepted, or declined. The high missing rate therefore points to a documentation gap rather than necessarily a practice gap. In a deployed workflow, ELSSA could help by prompting supervisors to confirm whether employee aftercare was offered in relevant cases, especially where the incident involves disorderly customers, collisions, threats, injury, or operator distress.
- **Moderately missing scene-resolution and injury-assessment actions require careful interpretation.** `VEHICLE_HOLD`, `INJURY_ASSESS`, and `SCENE_CLEAR` show lower but still substantial missing rates of 58.5%, 54.1%, and 46.5%, respectively. These codes illustrate the difference between documenting an outcome and documenting the procedural action. For instance, an SOR may state that there were no injuries, that the operator was fit to continue, or that service resumed, without explicitly stating that injuries were assessed or the scene was cleared.

Taken together, Fig. 15 refines the interpretation of missing action codes. The dominant missing codes are not primarily service-control actions; they are scene-management, investigation, safety, and support actions that are conditional and often implicit in short SOR narratives. This distinction is important for how ELSSA should be used in practice. The system should not automatically treat an undocumented item as a confirmed procedural failure. Instead, high-missing codes identify where the report may benefit from targeted clarification. In a drafting workflow, ELSSA could ask supervisors to confirm whether passengers were managed, whether the scene was secured, whether witnesses were interviewed, whether PPE was used, whether injury assessment was performed, and whether EFAP or other aftercare was offered. Such prompts would improve the completeness and auditability of incident records while preserving the supervisor's authority over the final report.

5. All-in-one: ELSSA—experience- and logic-integrated system for supervisor assistance

Building on the rule-based hierarchy-aware RAG and the experience-based GraphRAG modules, we integrate both into a single interactive LLM application, **ELSSA**.[2] As illustrated in Fig. 16, ELSSA provides a unified interface through which supervisors can obtain incident-management guidance grounded simultaneously in (i) formal TTC SOPs ("how the book says to do it") and (ii) historical incident experience ("how similar situations were handled in practice"). The system is designed to be evidence-traceable, configurable to agency preferences, and usable in conversational form.

[2] A prototype implementation is available on GitHub upon request.

At query time, a front-end LLM agent receives the supervisor's description of the active incident, such as "bus–auto collision with minor damage, no injuries, detour in effect," and formulates targeted sub-queries for each backend. One sub-query is routed to the hierarchy-aware SOP RAG module to retrieve formal procedures, prerequisites, and safety cautions. A second sub-query is routed to the experience-based knowledge graph to identify structurally similar historical incidents and their associated action strategies. The agent then synthesizes the two evidence streams into a structured recommendation that explicitly separates handbook-prescribed actions from experience-based observations. For example, the response may first list required SOP steps, then indicate how similar incidents were historically handled, and finally summarize likely operational implications such as service gaps, vehicle holding, passenger transfer, or out-of-service duration. When the SOP-grounded recommendation and the experience-based recommendation diverge, the system should expose this discrepancy rather than resolve it automatically. In such cases, the SOP module provides the procedural baseline, while the experience module provides operational precedent. The discrepancy itself may be useful to the supervisor, because it identifies where past practice may differ from formal guidance or where the current incident may require additional judgment.

Human-in-the-loop validation is therefore not an optional add-on, but a necessary condition for practical deployment. Human expertise is required at multiple stages of the system lifecycle. During system development, domain experts are needed to validate the SOP hierarchy, review the action-code scheme, adjudicate ambiguous historical records, and identify recommendations that may be linguistically plausible but operationally inappropriate. During deployment, supervisors should use ELSSA as an evidence-organizing and checklist-support tool rather than as a prescriptive authority. During system maintenance, expert feedback can be used to correct extraction errors, update obsolete procedures, improve the knowledge graph, and calibrate the recommendation policy. This is especially important for rare, high-consequence, or out-of-distribution incidents, where historical analogues may be sparse, misleading, or unavailable.

The integrated design also provides a foundation for incorporating more precise computational decision aids. In the present framework, the SOP module supplies the procedural structure of the response, while the experience-based module enriches that structure with concrete examples from prior incidents. Future extensions could add a compute-aided module that uses GTFS schedules, vehicle locations, timetable, and other constraints to estimate *quantitative* consequences of alternative actions, such as expected gap size, passenger delay, clearance time, short-turn feasibility, or diversion impact. Such a module would address the "last-mile" decision problem by complementing text-based reasoning with operational calculations.

To support multi-turn use, ELSSA maintains a compact memory buffer containing the active incident description, retrieved SOP evidence, relevant historical examples, and user-provided clarifications. Follow-up questions can therefore be answered by combining stored case context with fresh retrieval when necessary. The supervisor-facing interface presents both the synthesized recommendation and the supporting evidence, allowing users to inspect how the response was derived. This design positions ELSSA not as a black-box oracle, but as a transparent, evidence-linked, and human-supervised decision-support assistant for incident-control reasoning.

6. Conclusion

This work introduces **ELSSA**, an Experience- and Logic-integrated System for Supervisor Assistance that complements optimisation-based incident control with *procedurally faithful, experience-grounded* guidance. ELSSA combines an agentic, hierarchy-aware RAG over the TTC Emergency Response Handbook with a knowledge-graph-based RAG over historical incident reports, and exposes both rule-based and experience-based evidence through an interactive LLM interface.

Empirically, ELSSA's hierarchy-aware RAG outperforms several strong RAG baselines on supervisor training quizzes, with the largest gains on rewritten questions that require preserving SOP structure rather than relying on flat chunk retrieval. The experience-based GraphRAG module provides stronger action-code recovery than conventional RAG variants across original, radio-style, and paraphrased incident descriptions. With lightweight auto-regularization, ELSSA further reduces unsupported action fabrication and achieves the strongest performance on the temporally separated 2023 held-out database. A compliance analysis shows that service recovery and core documentation fields are highly visible in service occurrence reports, while conditional scene-management, safety, medical, and aftercare actions are often under-narrated. This finding suggests that ELSSA is most appropriately viewed as a checklist-style and evidence-linking assistant, rather than as an autonomous judge of supervisor compliance.

Beyond the TTC case study, our findings suggest broader design principles for LLM-based decision support in high-stakes operations: preserving document hierarchy improves the fidelity of rule-based retrieval; structured metadata and knowledge graphs can substantially enhance retrieval coverage in small, domain-specific corpora; simple statistical regularisers can stabilise LLM-extracted graphs; and agentic early stopping helps keep evidence compact and focused under tight latency and context constraints.

There are several limitations in the present study that motivate future work.

- **Richer and broader experience bases.** While the initial study and the supplementary longitudinal dataset validate the system's viability, they remain limited to specific routes, single transport modes, and a restricted subset of incident types. Future work must extend the incident knowledge graph to incorporate multi-year, multi-modal data to validate the system's robustness in handling complex edge cases, seasonal variations, and cross-route generalization.
- **Comparison with traditional decision-support systems.** A more complete evaluation should compare ELSSA with traditional decision-support approaches, such as rule-based expert systems, and dashboard-based control-room decision aids. Such comparisons would clarify whether the system provides incremental value beyond established decision-support paradigms.
- **Human-centric validation and practical decision metrics.** While this study establishes the efficacy of ELSSA from an informatics perspective (focusing on RAG-enhanced hit rates and accuracy), evaluating true operational impact requires a shift toward a

decision-making paradigm. A notable limitation of the current methodology is the reliance on automated LLM judges to compute compliance metrics. Although Inter-Rater Reliability (IRR) between dual LLM evaluators was reported to strengthen the evaluation, this approach remains susceptible to well-documented model artifacts, including verbosity and self-enhancement biases. Future research must prioritize human-in-the-loop validation involving experienced domain experts to establish a definitive ground truth. Furthermore, subsequent pilot deployments should evaluate practical operational indicators—such as reductions in response latency, cognitive workload, and mitigation efficacy—under agency-specific governance.

- **Advanced reasoning paradigms and DSS integration.** Due to current scope limits, this study focused heavily on optimizing information acquisition via advanced RAG design rather than performing direct comparative analyses with traditional, non-LLM Decision Support Systems (DSS). Moving forward, ELSSA's cognitive framework can be significantly enhanced by implementing advanced prompting strategies, such as Manual Few-shot Chain-of-Thought (CoT), to refine decision paths and rigorously reduce contradictory logic in highly complex scenarios. Additionally, future iterations will explore decision-focused LLM fine-tuning and hybrid architectures that integrate ELSSA's contextualized explanations and checklist safeguards directly with traditional optimization-based control tools (e.g., headway management or diversion planning algorithms).
- **Latency-aware orchestration for rule-based SOP retrieval.** Although the hierarchy-aware SOP RAG module improves rule-based question answering, its agent-in-the-loop retrieval and early-stop design requires more LLM calls. Future work should investigate latency-aware orchestration strategies for rule-based retrieval, such as adaptive early-stop policies, caching frequently retrieved SOP sections, and replacing some LLM-based routing decisions with smaller classifiers or deterministic heuristics.

Taken together, these results indicate the proposed methods can improve the relevance, traceability, and robustness of LLM-supported incident-management assistance. ELSSA provides a foundation for evidence-linked and human-supervised decision support in transit operations, while broader data coverage, expert validation, and decision-outcome evaluation remain necessary before stronger claims can be made about network-wide operational impact.

CRediT authorship contribution statement

Jiahao Wang: Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis, Data curation, Conceptualization; **Amer Shalaby:** Writing – review & editing, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization.

Data availability

The data that has been used is confidential.

Appendix A. Algorithm pseudo-code for auto-regularization of relation instances and entity-type inference

This appendix provides the complete pseudo-code for the frequency-based auto-regularization procedure described in the main text. As summarized in Algorithm 1, the procedure regularizes relation directions, imputes missing endpoint types, and infers initially unlabeled entity types through majority voting.

Algorithm 1: Auto-regularization of relation instances and entity-type inference.

Input: Entity table $\mathcal{E}$ with fields (id, type); relation table $\mathcal{R}$ with fields (src, tgt, desc); minimum vote threshold m.
Output: Updated $\mathcal{E}$ and $\mathcal{R}$ with regularized directions and completed types.

Step 0: attach entity types to relation endpoints
foreach $r \in \mathcal{R}$ **do**
  r.srcType $\leftarrow \mathcal{E}[r.\text{src}]$.type;
  r.tgtType $\leftarrow \mathcal{E}[r.\text{tgt}]$.type;
Step 1: canonical type signatures per description
foreach *relation description* d *appearing in* $\mathcal{R}$ **do**
  $S_d \leftarrow \{r \in \mathcal{R} \mid r.\text{desc} = d\}$;
  canonicalSrc$[d] \leftarrow$ most frequent non-null r.srcType in S_d;
  canonicalTgt$[d] \leftarrow$ most frequent non-null r.tgtType in S_d;
Step 2: canonical direction per (description, unordered type pair)
foreach $r \in \mathcal{R}$ *with known* r.srcType *and* r.tgtType **do**
  $p \leftarrow \{\{r.\text{srcType}, r.\text{tgtType}\}\}$; `// unordered type pair`
  count$[r.\text{desc}, p, (r.\text{srcType}, r.\text{tgtType})] \leftarrow$ count$[r.\text{desc}, p, (r.\text{srcType}, r.\text{tgtType})] + 1$;
foreach *key* (d, p) *in* count **do**
  canonicalDir$[d, p] \leftarrow \arg\max_{(s,t)}$ count$[d, p, (s, t)]$;
Step 3: straighten directions to match canonicalDir
foreach $r \in \mathcal{R}$ *with known* r.srcType *and* r.tgtType **do**
  $p \leftarrow \{\{r.\text{srcType}, r.\text{tgtType}\}\}$;
  if $(r.\text{desc}, p) \in$ canonicalDir **then**
    $(s^\star, t^\star) \leftarrow$ canonicalDir$[r.\text{desc}, p]$;
    if $(r.\text{srcType}, r.\text{tgtType}) = (t^\star, s^\star)$ **then**
      swap r.src and r.tgt; swap r.srcType and r.tgtType;
Step 4: impute missing endpoint types from canonical signatures
foreach $r \in \mathcal{R}$ **do**
  $d \leftarrow r$.desc;
  if r.tgtType *is null and* canonicalTgt$[d]$ *not null* **then**
    r.tgtType $\leftarrow$ canonicalTgt$[d]$;
  if r.srcType *is null and* canonicalSrc$[d]$ *not null* **then**
    r.srcType $\leftarrow$ canonicalSrc$[d]$;
Step 5: infer entity types by voting over relations
Initialize votes$[e, \tau] \leftarrow 0$ for all entities e and types τ;
foreach $r \in \mathcal{R}$ **do**
  if r.srcType *not null* **then**
    votes$[r.\text{src}, r.\text{srcType}] \leftarrow$ votes$[r.\text{src}, r.\text{srcType}] + 1$;
  if r.tgtType *not null* **then**
    votes$[r.\text{tgt}, r.\text{tgtType}] \leftarrow$ votes$[r.\text{tgt}, r.\text{tgtType}] + 1$;
foreach $e \in \mathcal{E}$ *with* e.type *null* **do**
  $(\tau^\star, c^\star) \leftarrow \arg\max_\tau$ votes$[e, \tau]$;
  if $c^\star \geq m$ **then**
    e.type $\leftarrow \tau^\star$;
return $\mathcal{E}, \mathcal{R}$;

Appendix B. LLM-as-judge evaluation prompt for experience-based suggestions

System prompt.

```
You are an impartial evaluator of transit-control decisions.
Score the CANDIDATE answer against the GOLD actions on these rubrics
(1-5 each, integer):
1. action_coverage    -- Does the CANDIDATE recommend the same operational
```

```
                          actions as GOLD?
                          5 = covers all main actions;
                          3 = partial;
                          1 = misses entirely.
2. resource_alignment -- Does it call for the same external resources
                          (tow, supervisor, EMS)?
3. impact_handling    -- Does it address the same service impacts
                          (gaps, OOS, etc.)?
4. safety             -- Does it avoid unsafe recommendations
                          (e.g., continue with mechanical issue)?
5. groundedness       -- Are recommendations supported by the EVIDENCE
                          rather than invented?
Return ONLY a JSON object:
{
    "action_coverage": int,
    "resource_alignment": int,
    "impact_handling": int,
    "safety": int,
    "groundedness": int,
    "comment": "<<=20 words>>"
}
```

User prompt template.

```
SITUATION:
{situation}
GOLD actions taken historically:
{gold_actions}
EVIDENCE shown to the candidate (summary):
{evidence_summary}
CANDIDATE answer:
{cand_str}
Return JSON only.
```

The variable `{situation}` denotes the incident or service-control context provided to the recommendation system; `{gold_actions}` denotes the historical actions recorded by the transit supervisor; `{evidence_summary}` denotes the retrieved evidence available to the candidate system; and `{cand_str}` denotes the generated recommendation being evaluated. The JSON-only output constraint was used to enable automatic parsing and aggregation of rubric-level scores.

Appendix C. Event counts by incident type and event type

The detailed distribution of records by incident type and event type is reported in Table C.1.

Table C.1
Event counts by incident type and event type.

Incident Type	Event Type	Count
Cleaning	General Cleaning	3
	Unsanitary/Sharps	9
General Delay	Construction	4
	Emergency	1
	Passenger Loads	1
	Special Events	1
	Traffic	6
Held By (Vehicle Not Involved)	Emergency Services	5
	Other External Causes	2
Investigation	Collision with Auto/Truck	19
	Collision with Moving Object	1
	Collision with Pedestrian	1
	Customer – Disorderly	4
	Customer – Illness	3
	Customer – Sleeper	1
	Damage to TTC Property	3
	Onboard Stationary	3
	Onboard in Motion	4
	Other	6
Late Leaving Garage/Yard	Issue with Vehicle	7
Mechanical – VISION	Destination Sign	1
	Early Departure – Error	1
	Vehicle Not Communicating/Tracking	1
Mechanical Issues	Articulation	3
	Body Interior (Stanchion/Panel/Seats)	4
	Brakes	2
	Compressed Air	1
	Door Master Seal	1
	Doors (including door master seal)	2
	Fluid Leak (coolant, fuel, hydraulic, etc.)	6
	HEV/Check/Stop Engine	3
	HVAC/Heat/Cool	3
	Lighting	1
	Mirrors	7
	No Start	1
	No/Low Accelleration	1
	Operator Seat (including seatbelt)	2
	Other	10
	Ramp	3
	Smoke/Fire	1
	Stalled	2
	Stop Request Chimes	1
	Transmission	2
	Trucks (noisy, wheel flange lubricator, etc.)	1
	Vehicle Controls/Dash (speedometer)	2
	Warning Light/Alarm Systems	5
Operator related	Irregularity	6
	No Operator	33
	Off Route (Bus only)	1
	Operator Ill	4
	Schedule Adherence – End of Line	5
	Schedule Adherence – Mid-route	1
Utilized Off Route	Shuttle – Emergency	13
	Supplement Service	3

Appendix D. Entity and relation schema for the example incident-level graph

The entities and relations extracted for the example incident-level knowledge graph are listed in Tables D.1 and D.2, respectively.

Table D.1
Entities in the example incident-level knowledge graph.

Entity type	Entity label
Incident pattern	MECHANICAL
Event category	FLUID_LEAK
Service impact	OUT_OF_SERVICE
Service impact	SERVICE_GAP
Root cause	REAR_BUS_LEAK
Action strategy	PUMP_CATCH_BASIN
Action strategy	NOTIFY_ENVIRONMENTAL_AGENCY
Action strategy	REPLACE_VEHICLE
Action strategy	OPERATOR_TO_RELIEF_POINT
Action strategy	PLACE_REPLACEMENT_VEHICLE
Action strategy	NOTIFY_MINISTRY_OF_ENVIRONMENT
Measurement	16_Min
Resource requirement	PUMPER_TRUCK
Resource requirement	RELIEF_OPERATOR
Resource requirement	REPLACEMENT_VEHICLE
Location pattern	MDGA
Location pattern	CATCH_BASIN

Table D.2
Relations between entities in the example incident-level knowledge graph.

Source	Relation	Target
FLUID_LEAK	causes	REAR_BUS_LEAK
FLUID_LEAK	happens_at	CATCH_BASIN
OPERATOR_TO_RELIEF_POINT	happens_at	MDGA
PLACE_REPLACEMENT_VEHICLE	happens_at	MDGA
PUMP_CATCH_BASIN	happens_at	CATCH_BASIN
REPLACE_VEHICLE	happens_at	MDGA
MECHANICAL	in_scope	FLUID_LEAK
SERVICE_GAP	quantifies	16_Min
PLACE_REPLACEMENT_VEHICLE	requires	REPLACEMENT_VEHICLE
PUMP_CATCH_BASIN	requires	PUMPER_TRUCK
REPLACE_VEHICLE	requires	RELIEF_OPERATOR
REPLACE_VEHICLE	requires	REPLACEMENT_VEHICLE
FLUID_LEAK	resolves	NOTIFY_MINISTRY_OF_ENVIRONMENT
FLUID_LEAK	resolves	PLACE_REPLACEMENT_VEHICLE
FLUID_LEAK	resolves	PUMP_CATCH_BASIN
FLUID_LEAK	resolves	REPLACE_VEHICLE
SERVICE_GAP	resolves	PLACE_REPLACEMENT_VEHICLE
FLUID_LEAK	results_in	OUT_OF_SERVICE
FLUID_LEAK	results_in	SERVICE_GAP

Appendix E. Complete action coding scheme

This appendix reports the complete action-coding scheme used to evaluate whether a generated recommendation contains the operational actions documented in the historical *Actions Taken* field. The codebook normalizes heterogeneous free-text incident narratives into a compact set of canonical action codes. Each code is assigned to an operational category and a facet. The facet distinguishes whether the code represents a supervisor action (*act*) or a documentation requirement (*doc*). The category overview is provided in Table E.1, while the complete codebook is reported in Table E.2 and Table E.3.

Table E.1
Overview of the revised action-coding categories.

Category	Description	# Codes
Communication & Investigation (COMM)	Contacting or updating Transit Control, radio communication, requesting emergency or operational support, and conducting incident investigation.	8
Scene Management (SCENE)	Attending, securing, holding, clearing, and managing the incident scene, vehicle, and affected passengers.	5
Safety & Medical Response (SAFETY)	Injury assessment, medical assistance, emergency response, PPE use, and infection-control actions when applicable.	5
Service Operations (SERVICE)	Service adjustment, gap or delay management, diversion, restoration, relief arrangement, and temporary stop management.	7
Evidence & Documentation (DOC)	Report selection and submission, as well as required documentation fields such as time, location, vehicle, employee, injury, damage, and witness information.	11
Support & Aftercare (SUPPORT)	Employee assistance, stress-related removal from duty, and non-medical customer support.	3

Table E.2
Complete revised action codebook.

Code	Cat.	Facet	Definition
TCC_NOTIFY	COMM	act	Report the incident to Transit Control / TCC, or acknowledge the initial Transit Control call.
TCC_UPDATE	COMM	act	Provide an updated assessment or situation report to Transit Control after the initial contact.
ASSIST_CALL	COMM	act	Broadcast an assist call, priority call, or equivalent urgent request for support.
RADIO_COMM	COMM	act	Use radio communication during the incident, including normal-status or routine operational communication.
EMS_REQUEST	COMM	act	Request police, fire, EMS, or 911. This captures the act of requesting emergency services, distinct from their subsequent attendance or treatment.
INVEST_CONDUCT	COMM	act	Conduct an on-scene investigation to establish what happened.
WITNESS_INTERVIEW	COMM	act	Interview the operator, customers, third parties, or witnesses to establish incident facts.
EVIDENCE_VIDEO	COMM	act	Preserve physical evidence or arrange CCTV/onboard video download.
SCENE_ATTEND	SCENE	act	Supervisor or cab proceeds to and arrives at the incident scene.
SCENE_SECURE	SCENE	act	Secure or protect the scene, such as blocking the area, using pylons, or preserving a crime/collision scene.
SCENE_CLEAR	SCENE	act	Clear the scene once the incident is resolved.
VEHICLE_HOLD	SCENE	act	Hold the bus or vehicle in place, for example pending supervisor arrival, investigation, or position marking.
PASSENGER_MGMT	SCENE	act	Offload, transfer, shelter, or otherwise manage customers on or near the affected vehicle.
INJURY_ASSESS	SAFETY	act	Check affected persons for injuries or offer an injury assessment.
MEDICAL_OFFER	SAFETY	act	Offer first aid, medical attention, or EMS to an affected person.
MEDICAL_PROVIDE	SAFETY	act	Provide first aid or document that EMS treated or transported the affected person.
PPE_USE	SAFETY	act	Use PPE such as a safety vest, gloves, safety glasses, or other protective equipment while attending or working at the scene.
INFECTION_CONTROL	SAFETY	act	Apply infection-control procedures, such as using nitrile gloves or minimizing contact with bodily fluids.
SERVICE_ADJUST	SERVICE	act	Make a general service adjustment in response to the disruption.
GAP_MANAGE	SERVICE	act	Manage the resulting headway gap, such as advancing runs, holding vehicles, short-turning, or otherwise adjusting spacing.
DELAY_RECORD	SERVICE	doc	Record the delay or out-of-service (OOS) duration caused by the incident.
DIVERSION_ROUTE	SERVICE	act	Implement a diversion or alternate routing around the incident.
RESTORE_SERVICE	SERVICE	act	Return the vehicle/operator to service or otherwise restore normal operation.
RELIEF_ARRANGE	SERVICE	act	Arrange a relief operator, change-off, or replacement vehicle.
TEMP_STOP_SETUP	SERVICE	act	Establish a temporary stop or relocate/flag a stop according to temporary-stop procedures.
REPORT_SELECT	DOC	act	Select and complete the appropriate report form, such as a Surface Supervisory Occurrence report.
REPORT_SUBMIT	DOC	act	Log and submit the required report(s) to the appropriate receiving departments.
TIME_RECORD	DOC	doc	Record relevant times, such as occurrence time, out-of-service start/end, and clearance time.
LOCATION_DETAIL	DOC	doc	Record the precise location or position of the incident.
VEHICLE_ID	DOC	doc	Record the bus, cab, vehicle, run, or unit identifier.

Table E.3
Continued...

Code	Cat.	Facet	Definition
EMPLOYEE_ID	DOC	doc	Record the operator, supervisor, badge, or employee identifier.
THIRD_PARTY_INFO	DOC	doc	Record third-party or other-vehicle information, such as owner, plate, licence, or insurance details.
INJURY_STATUS	DOC	doc	Document whether anyone was injured and, where applicable, the injury outcome.
EXTERNAL_FACTORS	DOC	doc	Record environmental or external factors, such as weather, lighting, road, or visibility conditions.
DAMAGE_DETAIL	DOC	doc	Describe the nature or extent of vehicle, infrastructure, or property damage.
WITNESS_CONTACT	DOC	doc	Record witness names and contact details for follow-up.
EFAP_OFFER	SUPPORT	act	Offer the Employee and Family Assistance Program (EFAP) or equivalent employee support.
STRESS_REMOVE	SUPPORT	act	Remove an operator from service, or document that the operator is unable to continue, due to stress, shock, or fitness-for-duty concerns.
CUSTOMER_ASSIST	SUPPORT	act	Provide non-medical assistance to a customer, such as mobility help, accommodation, or other customer support.

Appendix F. Action codes used in the experience-based evaluation

This appendix summarizes the canonical action-code taxonomy used in the experience-based evaluation. The taxonomy consolidates semantically equivalent surface expressions observed in historical incident records into a controlled set of reference action codes.

Table F.1 summarizes the six operational categories and the number of codes assigned to each category. Table F.2 lists the codes for service and schedule management. Table F.3 presents the codes for vehicle disposition and cab, supervisor, and scene management. Table F.4 reports the codes for communication and notification, medical and safety response, and operator and customer management.

Table F.1
Summary of action-code categories used in the experience-based evaluation.

Category	Number of codes
Service and schedule management	13
Vehicle disposition	9
Cab, supervisor, and scene management	4
Communication and notification	4
Medical and safety response	3
Operator and customer management	4
Total	**37**

Table F.2
Canonical action codes for service and schedule management. Italicized expressions indicate common surface variants merged into the corresponding canonical code.

Canonical code	Operational meaning
Service and schedule management	
SERVICE_ADJUSTMENT	General service or schedule adjustment intended to restore regular headways.
CHANGE_OFF	Operator or vehicle change-off, including relief swaps.
EXPRESS_SERVICE_DEPLOYMENT	Deploy express service to close a service gap or reduce delay propagation.
PLACE_FOR_SERVICE	Place a bus at a specified time or location to fill service. *Includes CIS and TCBO placement variants.*
ADVANCE_RUN	Advance a run to support service recovery. *Includes advanced-for-service variants.*
DETOURDIVERSION	Divert or detour buses around an obstruction, incident, or blocked segment.
SHORT_TURN	Short-turn a vehicle to recover schedule regularity or rebalance service.
RETURN_TO_ROUTE	Return a previously diverted bus to its regular route.
DEAD_HEAD	Reposition a vehicle out of service.
AUTHORIZED_EARLY_DEPARTURE	Authorize an operator to depart earlier than scheduled.
FOLLOWING_RUN_ADVISED	Advise the following run of the incident, service condition, or required action.
RELIEF_OPERATOR_PLACED	Place a relief operator to support continued service operation.
GAP_MGMT	Manage a specific service gap, such as an observed or anticipated headway gap.

Table F.3
Canonical action codes for vehicle disposition and scene management. Italicized expressions indicate common surface variants merged into the corresponding canonical code.

Canonical code	Operational meaning
Vehicle disposition	
OPERATE_PRIVATE_TO_GARAGE	Operate the vehicle private, or out of service, to the garage or division.
TOW_BUS	Arrange a tow or tow truck for a disabled vehicle. *Includes arrange-tow variants.*
BREAK_MASTER_SEAL	Break the door master seal when required by the incident response.
ACTIVATE_YELLOW_ALARM	Activate or push the yellow emergency alarm.
OUT_OF_SERVICE	Place the vehicle out of service.
SHUT_DOWN	Shut down the bus.
SECURE_BUS	Secure the bus at the incident scene or designated location.
SECURE_VAULT	Secure the fare vault.
RETURN_TO_DIVISION	Return the vehicle or operator to the division.
Cab, supervisor, and scene management	
HOLD_FOR_CAB	Hold the bus for cab or supervisor attendance. *Includes holding-for-cab variants.*
BUS_HOLD	Hold the bus for investigation, police attendance, or other scene-management needs.
MOBILE_SUPERVISOR	Dispatch a mobile supervisor to attend the scene.
EMERGENCY_SHUTTLE	Deploy an emergency shuttle to maintain passenger movement during disruption.

Table F.4
Canonical action codes for communication, safety response, and operator or customer management. Italicized expressions indicate common surface variants merged into the corresponding canonical code.

Canonical code	Operational meaning
Communication and notification	
ADVISE_TRANSIT_CONTROL	Advise or notify Transit Control. *Includes TCC-advised variants.*
NOTIFY_SITUATION	Notify the relevant party of the incident or operating condition. *Includes notified variants.*
ADVISE_SUPERVISOR	Advise the supervisor of the incident or required follow-up.
ADVISE_GARAGE	Advise the garage or division.
Medical and safety response	
OFFER_MEDICAL/HOLD_FOR_EMS	Offer medical assistance or hold the vehicle for EMS attendance.
TRANSPORT_OPERATOR	Transport the operator, including transport to the division.
TRANSFERRED_TO_HOSPITAL	Record that a customer or operator was transferred to hospital.
Operator and customer management	
CLEAR_TO_CONTINUE	Clear the operator to continue service. *Includes cleared variants.*
COUNSELING	Counsel the operator. *Includes counsel-operator variants.*
SCHEDULE_ADHERENCE_REMINDER	Remind the operator about schedule adherence.
EDUCATE_CUSTOMER	Educate or inform the customer.

References

Abbas, M., Ferrari, A., Shatnawi, A., Enoiu, E., Saadatmand, M., Sundmark, D., 2023. On the relationship between similar requirements and similar software: a case study in the railway domain. Requir. Eng. 28 (1), 23–47.

Ai, G., Zuo, X., Chen, G., Wu, B., 2022. Deep reinforcement learning based dynamic optimization of bus timetable. Appl. Soft Comput. 131, 109752.

Ai, G., Zuo, X., Chen, G., Zhou, M., Wu, B., Zhao, X., 2026. XRL-TO: an explainable reinforcement learning-based approach for bus timetable dynamic optimization. Expert Syst. Appl., 297, 129271.

Ashqar, H.I., Jaber, A., Alhadidi, T.I., Elhenawy, M., 2024. Advancing object detection in transportation with multimodal large language models (MLLMs): a comprehensive review and empirical testing. arXiv:2409.18286.

Atkinson, C., Choi, T., Sabol, P., Flynn, D., Hicks, J., Englin, E., Drake, J., Rittmuller, R.J., 2024. Summary of RFI Responses—Opportunities and Challenges of Artificial Intelligence (AI) in Transportation. Report. U.S. Department of Transportation, Volpe National Transportation Systems Center; Advanced Research Projects Agency—Infrastructure (ARPA-I). December 2024.

Boyer, V., Ibarra-Rojas, O.J., Ríos-Solís, Y.Á., 2018. Vehicle and crew scheduling for flexible bus transportation systems. Transp. Res. B Methodol. 112, 216–229.

Ceder, A., 2016. Public Transit Planning and Operation: Modeling, Practice and Behavior. CRC press.

Consilvio, A., Calabrò, L., Di Febbraro, A., Sacco, N., 2021. A multimodal solution approach for mitigating the impact of planned maintenance on metro rail attractiveness. EURO J. Transp. Logist. 10, 100047.

Devunuri, S., Qiam, S., Lehe, L.J., 2024. ChatGPT for GTFS: benchmarking LLMs on GTFS semantics… and retrieval. Public Transp. 16 (2), 333–357.

Du, Y., 2024. Large models in transportation infrastructure: a perspective. Intell. Transp. Infrastruct. 3, liae007.

Dwivedi, Y.K., Kshetri, N., Hughes, L., Slade, E.L., Jeyaraj, A., Kar, A.K., Baabdullah, A.M., Koohang, A., Raghavan, V., Ahuja, M., et al., 2023. Opinion paper:"so what if chatGPT wrote it?" Multidisciplinary perspectives on opportunities, challenges and implications of generative conversational AI for research, practice and policy. Int. J. Inf. Manag. 71, 102642.

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R.O., Larson, J., 2024. From local to global: a graph RAG approach to query-focused summarization. arXiv:2404.16130.

Feng, G., Xu, P., Cui, D., Dai, X., Liu, H., Zhang, Q., 2021. Multi-stage timetable rescheduling for high-speed railways: a dynamic programming approach with adaptive state generation. Complex Intell. Syst. 7 (3), 1407–1428.

Heidariharehsoo, N., Saidi, S., 2023. Exploring alternative bus operations strategies for adaptive and efficient transit systems in the face of disruptions. J. Public Transp. 25, 100060.

International Transport Forum (ITF), 2025. AI for Transport Authorities: Principles and Practical Guidance. OECD Publishing, Paris.
Itani, A., Srikukenthiran, S., Shalaby, A., 2020. Capacity-constrained bus bridging optimization framework. Transp. Res. Rec. 2674 (5), 600–612.
Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al., 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In Advances in Neural Information Processing Systems vol. 33, 9459–9474.
Li, S., Wang, S., Xu, G., Wu, L., 2025. A model predictive control framework for urban rail transit signal optimization with time-series passenger flow forecasting. Informatica 49 (5), 311–324.
Liang, L., Bo, Z., Gui, Z., Zhu, Z., Zhong, L., Zhao, P., Sun, M., Zhang, Z., Zhou, J., Chen, W., et al., 2025. KAG: boosting LLMs in professional domains via knowledge augmented generation. In: Companion Proceedings of the ACM on Web Conference 2025, pp. 334–343.
Mahmoudi, R., Saidi, S., Emrouznejad, A., 2025. Mathematical programming in public bus transit design and operations: emerging technologies and sustainability–a review. Socio-Econ. Plan. Sci. 98, 102155.
Matsumoto, N., Moran, J., Choi, H., Hernandez, M.E., Venkatesan, M., Wang, P., Moore, J.H., 2024. KRAGEN: a knowledge graph-enhanced RAG framework for biomedical problem solving using large language models. Bioinformatics 40 (6), btae353.
Mortaheb, M., Khojastepour, M. A.A., Chakradhar, S.T., Ulukus, S., 2025. Re-ranking the context for multimodal retrieval augmented generation. arXiv:2501.04695 .
Papageorgiou, E., Chronis, C., Varlamis, I., Himeur, Y., 2024. A survey on the use of large language models (LLMs) in fake news. Future Internet 16 (8), 298.
Prajapati, S., Singh, T., Hegde, C., Chakraborty, P., 2024. Evaluation and comparison of visual language models for transportation engineering problems. arXiv:2409.02278.
Rane, N.L., 2023. Multidisciplinary collaboration: key players in successful implementation of ChatGPT and similar generative artificial intelligence in manufacturing, finance, retail, transportation, and construction industry. OSF Preprints. https://doi.org/10.31219/osf.io/npm3d.
Roberts, J., Lüddecke, T., Das, S., Han, K., Albanie, S., 2023. GPT4GEO: how a language model sees the world's geography. arXiv:2306.00020.
Rodriguez, J., Koutsopoulos, H.N., Zhao, J., 2025. Deploying robust decision support systems for transit headway control: rider impacts, human factors and recommendations for scalability. arXiv:2509.08231.
Roy, N., Ribeiro, L. F.R., Blloshmi, R., Small, K., 2024. Learning when to retrieve, what to rewrite, and how to respond in conversational QA. arXiv:2409.15515.
Syed, U., Light, E., Guo, X., Zhang, H., Qin, L., Ouyang, Y., Hu, B., 2024. Benchmarking the capabilities of large language models in transportation system engineering: accuracy, consistency, and reasoning behaviors. arXiv:2408.08302.
Traag, V.A., Waltman, L., Van Eck, N.J., 2019. From Louvain to Leiden: guaranteeing well-connected communities. Sci. Rep. 9 (1), 1–12.
Vake, D., Vičič, J., Tošić, A., 2025. Bridging the question-answer gap in retrieval-augmented generation: hypothetical prompt embeddings. IEEE Access 13, 129952–129961.
Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. Adv. Neural Inf. Process. Syst. 30, 5998–6008.
Wang, C., Shi, H., Zuo, X., 2020. A multi-objective genetic algorithm based approach for dynamical bus vehicles scheduling under traffic congestion. Swarm Evol. Comput. 54, 100667.
Wang, J., Shalaby, A., 2024. Transit pulse: utilizing social media as a source for customer feedback and information extraction with large language model. arXiv:2410.15016.
Wang, P., Wei, X., Hu, F., Han, W., 2024. TransGPT: multi-modal generative pre-trained transformer for transportation. In: 2024 International Conference on Computational Linguistics and Natural Language Processing (CLNLP). IEEE, pp. 96–100.
Wu, J., Zhu, J., Qi, Y., Chen, J., Xu, M., Menolascina, F., Grau, V., 2024. Medical graph RAG: towards safe medical large language model via graph retrieval-augmented generation. arXiv:2408.04187.
Xu, D., Chen, W., Peng, W., Zhang, C., Xu, T., Zhao, X., Wu, X., Zheng, Y., Chen, E., 2023. Large language models for generative information extraction: a survey. arXiv:2312.17617.
Xu, S., Chen, M., Chen, S., 2024a. Enhancing retrieval-augmented generation models with knowledge graphs: innovative practices through a dual-pathway approach. In: International Conference on Intelligent Computing. Springer, pp. 398–409.
Xu, S., Wu, Z., Zhao, H., Shu, P., Liu, Z., Liao, W., Li, S., Sikora, A., Liu, T., Li, X., 2024b. Reasoning before comparison: LLM-enhanced semantic similarity metrics for domain specialized text analysis. arXiv:2402.11398.
Yan, X.J., Hunter, M., 2023. Promoting Equitable AI Applications in Transportation. Technical Report Project P6. Southeastern Transportation Research, Innovation, Development and Education Center (STRIDE), University of Florida Transportation Institute. Gainesville, FL. Funding Agreement No. 69A3551747104; 102 pages.
Zdravković, S., Dobrić, F., Injac, Z., Lukić-Vujadinović, V., Veličković, M., Bursać Vranješ, B., Marinković, S., 2025. AI-driven safety evaluation in public transport: a case study from belgrade's closed transit systems. Sustainability 17 (18), 8283.
Zhang, F., Zhu, D., Ming, J., Jin, Y., Chai, D., Yang, L., Tian, H., Fan, Z., Chen, K., 2025. DH-RAG: a dynamic historical context-powered retrieval-augmented generation method for multi-turn dialogue. arXiv:2502.13847.
Zhang, J., Ilievski, F., Ma, K., Kollaa, A., Francis, J., Oltramari, A., 2023a. A study of situational reasoning for traffic understanding. In: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 3262–3272.
Zhang, W., Deng, Y., Liu, B., Pan, S.J., Bing, L., 2023b. Sentiment analysis in the era of large language models: a reality check. arXiv:2305.15005.
Zheng, O., Abdel-Aty, M., Wang, D., Wang, C., Ding, S., 2023. TrafficsafetyGPT: tuning a pre-trained large language model to a domain-specific expert in transportation safety. arXiv:2307.15311.
Zhong, Q., Ding, L., Liu, J., Du, B., Tao, D., 2023. Can chatGPT understand too? A comparative study on ChatGPT and fine-tuned BERT. arXiv:2302.10198.
Zhu, Z., Hu, T., Zhang, H., Yang, D., Chen, H., Zhang, M., Chen, X., 2025. Conversational intent-driven graphRAG: enhancing multi-turn dialogue systems through adaptive dual-retrieval of flow patterns and context semantics. arXiv:2506.19385.